

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение высшего образования
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»

ДОПУСТИТЬ К ЗАЩИТЕ

Заведующий кафедрой № 43

профессор, д-р техн. наук

должность, уч. степень, звание

подпись, дата

М. Ю. Охтилев

инициалы, фамилия

БАКАЛАВРСКАЯ РАБОТА

на тему

Оптимизация распределения ресурсов методом

динамического программирования

выполнена

Дудко Никитой Ивановичем

фамилия, имя, отчество студента в творительном падеже

по направлению подготовки

02.03.03

код

Математическое обеспечение

наименование направления

и администрирование информационных систем

наименование направления

направленности

02

код

Системный анализ в информационных

наименование направленности

технологиях

наименование направленности

Студент группы №

4036

подпись, дата

Н.И.Дудко

инициалы, фамилия

Руководитель

проф., д-р техн. наук, доцент

должность, уч. степень, звание

подпись, дата

С.И.Колесникова

инициалы, фамилия

Санкт-Петербург 2024

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение высшего образования
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»

УТВЕРЖДАЮ

Заведующий кафедрой № 43

профессор, д-р техн. наук

должность, уч. степень, звание

М. Ю. Охтилев

подпись, дата

инициалы, фамилия

ЗАДАНИЕ НА ВЫПОЛНЕНИЕ БАКАЛАВРСКОЙ РАБОТЫ

студенту группы

4036

номер

Дудко Никите Ивановичу

фамилия, имя, отчество

на тему

Оптимизация распределения ресурсов методом

динамического программирования

утвержденную приказом ГУАП от

№

Цель работы:

автоматизация решения задачи оптимального инвестирования

на основе динамического программирования

Задачи, подлежащие решению:

формализация задачи оптимального распределения

инвестиций между предприятиями; проектирование и разработка программы, использующей в

своей основе метод динамического программирования; тестирование и исследование ПО

Содержание работы (основные разделы):

теоретические основы и методология
динамического программирования;

практическая часть;

программное решение задачи оптимизации распределения ресурсов на основе метода
динамического программирования;

исследование эффективности работы алгоритма динамического программирования;

Срок сдачи работы « ____ »

20 ____

Руководитель

профессор, д-р техн. наук

должность, уч. степень, звание

С.И.Колесникова

подпись, дата

инициалы, фамилия

Задание принял(а) к исполнению

студент группы №

4036

подпись, дата

Н.И.Дудко

инициалы, фамилия

РЕФЕРАТ

Тема ВКР: «Оптимизация распределения ресурсов на основе метода динамического программирования»

Ключевые слова: динамическое программирование, симплекс метод, генетический алгоритм, метод ветвей и границ, жадный алгоритм, сравнение эффективности, оптимизационная задача, инвестиции, распределение ресурсов.

Актуальность: необходимость в поиске оптимальных способов принятия решений в условиях ограниченности денежных ресурсов; сложность правильного выбора алгоритма для решения задач среднесрочного планирования.

Цель данной работы заключается в разработке программного обеспечения для автоматизации решения задачи оптимального распределения инвестиционных ресурсов с использованием метода динамического программирования.

Задачи работы: формализация задачи оптимального распределения ресурсов в контексте инвестиционной деятельности; проектирование и разработка программного обеспечения, реализующего метод динамического программирования; исследование результатов решения задачи в зависимости от начальных данных и сравнение эффективности предложенного алгоритма с его аналогами.

Основные результаты: исследовано применение метода динамического программирования к задаче распределения ресурсов с целью минимизации времени завершения расчета и максимизации прибыли. Тестирование показало, что предложенный алгоритм превосходит аналогичные методы по скорости/точности при разумных сценариях использования.

ФИО студента: Дудко Никита Иванович.

Учебное заведение и группа: Государственный университет
аэрокосмического приборостроения, группа 4036.

ВКР содержит пояснительную записку в количестве 110 страниц, 8
таблиц и 30 рисунков.

ABSTRACT

Topic of the FQW: "Optimization of resource allocation based on the dynamic programming method"

Keywords: dynamic programming, simplex method, genetic algorithm, branch and boundary method, greedy algorithm, efficiency comparison, optimization task, investments, resource allocation.

Relevance: the need to find optimal ways to make decisions in conditions of limited monetary resources; the difficulty of choosing the right algorithm for solving medium-term planning problems.

The purpose of this work is to develop software for automating the solution of the problem of optimal allocation of investment resources using the dynamic programming method.

The tasks of the work: formalization of the problem of optimal allocation of resources in the context of investment activity; design and development of software implementing the dynamic programming method; investigation of the results of solving the problem depending on the initial data and comparison of the effectiveness of the proposed algorithm with its analogues.

The main results: the application of the dynamic programming method to the problem of resource allocation is investigated in order to minimize the calculation completion time and maximize profit. Testing has shown that the proposed algorithm is superior to similar methods in terms of speed/accuracy under reasonable use cases.

Student's full name: Nikita Ivanovich Dudko

Educational university and group: State University of Aerospace Instrumentation, group 4036

The FQW contains an explanatory note in the amount of 110 pages, 8 tables and 30 figures.

СОДЕРЖАНИЕ

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ.....	7
ВВЕДЕНИЕ	8
1. Теоретические основы и методология динамического программирования	13
1.1. Введение в метод динамического программирования	13
1.2. Методология решения на основе рекуррентных уравнений	15
1.3. Практическое применение метода динамического программирования...	17
1.4. Заключение по теоретическим основам.....	19
2. Практическая часть	20
2.1. Формализация задачи об оптимизации распределения ресурсов.....	20
2.2. Пример оптимизации распределения ресурсов на практике	25
2.3. Структурная схема задачи об оптимизации распределения ресурсов.	28
2.4. Заключение по практической части.....	29
3. Программное решение задачи оптимизации распределения ресурсов на основе метода динамического программирования.....	30
3.1. Подход к реализации ПО	30
3.2. Стек технологий ПО	31
3.3. Блок – схема алгоритма реализации ДП.....	44
3.4. Тестирование	46
4. Исследование эффективности работы алгоритма динамического программирования	52
4.1. Краткое введение в существующие альтернативы	52
4.2. Методология исследования	58
4.3. Проведение исследования	61
4.4. Вывод по проведенным исследованиям.....	76
ЗАКЛЮЧЕНИЕ	78
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	79
ПРИЛОЖЕНИЕ. Текст программы.....	82

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

DEAP – Distributed Evolutionary Algorithms in Python, библиотека Python для работы с генетическими алгоритмами

GUI – Графический интерфейс

IDE – Среда разработки

MVP – Minimum Viable Product

ОС – Операционная система.

UC – Use Case, форма описания требований к ПО

VS – редактор кода Visual Studio

ДП – Динамическое программирование

ЛП – Линейное программирование

ПО – Программное обеспечение.

ТПР – Теория принятия решений

ЦП –Целочисленное программирование

ВВЕДЕНИЕ

Задача оптимизации распределения ресурсов тесно связана с задачей инвестирования, которая, в свою очередь, является одной из ключевых составляющих успешного развития любого бизнеса в современном мире. Грамотное распределение ресурсов и принятие оптимальных инвестиционных решений позволяет компаниям максимизировать прибыль, минимизировать риски и обеспечить устойчивый рост в долгосрочной перспективе. В настоящее время это является наиболее важной задачей для нефтегазовой добывающей промышленности Российской Федерации, поскольку возникли проблемы доступа к внешнему финансированию из-за установленных санкций [1]. Данный факт делает невозможным привлечение инвестиций от европейских и американских компаний. Также имеется затруднение в получении банковских кредитов для поиска и разведки месторождений из-за высоких инвестиционных рисков [2].

Вследствие этого, возникает необходимость в поиске оптимальных способов принятия решений в области инвестиционной деятельности в условиях ограниченности ресурсов. Решение проблемы подобного класса обычно основывается на теории принятия решений (ТПР). «Эта область математики объединяет способы и процедуры формализации процесса принятия решений, трансформируя проблему выбора в математическую задачу и предлагая способы ее решения с использованием доступных методов» [3].

«Для задач ТПР в экономике решение принимается на основе конкретных исходных данных. Все параметры являются детерминированными. Выбор методов существенно зависит от общей постановки и применяемых методов решения» [3]. На основании существующих подходов к решению задач и представленной классификации в работе [3] Д. С. Набатовой можно представить следующую общую

классификацию задач теории принятия решений.

1. Задачи оперативного управления.

Задачи оперативного управления связаны с принятием решений в условиях текущей деятельности организации или системы, имеют детерминированные параметры и предполагают оптимизацию в краткосрочной перспективе. Решением таких задач является векторное значение, определяющее оптимальные параметры для данного момента времени, учитывающее ограничения ресурсов.

Такие задачи имеют следующие характеристики.

- Краткосрочный горизонт планирования: решения принимаются на короткие промежутки времени (от часов до нескольких дней).
- Высокая динамика: ситуация может быстро меняться, требуя оперативного реагирования.
- Наличие реальных ограничений: ресурсы и время ограничены, что требует быстрого и эффективного использования имеющихся средств.

Методы решения: «Линейное и квадратичное программирование, метод потенциалов для транспортных задач, методы целочисленного программирования (метод Гомори, метод ветвей и границ»[3].

Примеры задач: оптимизация объемов производства, оптимизация инвестиций, управление производственными процессами, распределение ресурсов в реальном времени, логистика и транспортировка.

2. Задачи перспективного и среднесрочного планирования и управления.

Эти задачи ориентированы на более длительные периоды времени и включают разработку стратегий и планов для обеспечения устойчивого развития системы. Параметры таких задач детерминированы, но изменяются во времени.

Характеристики.

- Среднесрочный и долгосрочный горизонт планирования:

временные рамки могут варьироваться от нескольких месяцев до нескольких лет.

- Стратегический характер: решения направлены на достижение долгосрочных целей и задач организации.

- Более детальный анализ и прогнозирование: требуются тщательные исследования и моделирование будущих сценариев.

Методы решения: динамическое программирование, принцип декомпозиции для сложных систем.

Примеры задач: стратегическое планирование развития компании, инвестиционное планирование, разработка маркетинговых стратегий, планирование научно-исследовательских и опытно-конструкторских работ.

3. Многокритериальные задачи принятия решений.

Эти задачи возникают в ситуациях, когда необходимо учитывать несколько критериев при принятии решения. Задачи этого типа имеют исторические корни и требуют применения специальных методов для нахождения компромиссных решений.

Характеристики.

- Наличие нескольких критериев: каждый из этих критериев может иметь разную важность и влиять на конечное решение.

- Конфликтующие цели: критерии могут быть взаимоисключающими, что усложняет процесс выбора оптимального решения.

- Неоднозначность и неопределенность: часто существует неопределенность в данных или будущих условиях, что требует применения специальных методов анализа.

- Необходимость компромиссов: решения должны учитывать баланс между различными критериями.

Методы решения: «Метод свертки (метод обобщенного критерия), метод приоритетов, метод уступок, метод идеальной точки. Специальные

методы: АНР, ELECTRE, MAUT, SMART»[3].

Примеры задач: выбор места для строительства объекта с учетом экологических, экономических и социальных факторов, принятие решений о портфеле инвестиций, оптимизация производственных процессов по нескольким параметрам (стоимость, качество, время).

4. Принятие решений в конфликтных ситуациях.

Эти задачи связаны с ситуациями, где присутствуют конкурирующие стороны с различными интересами и целями. Принятие решений в таких условиях требует учета стратегического поведения всех участников.

Характеристики.

- Конфликт интересов: наличие противоборствующих сторон, каждая из которых стремится максимально удовлетворить свои интересы.
- Переговорный процесс: часто требуется вести переговоры и искать взаимовыгодные решения.
- Стратегическое поведение: стороны могут предпринимать стратегические действия для достижения своих целей, включая тактику, уловки и манипуляции.
- Риск и неопределенность: высокий уровень неопределенности и риска, так как поведение других сторон может быть непредсказуемым.

Методы решения: методы теории игр.

Примеры задач: переговоры о заключении контракта, управление конфликтами в организации, разработка стратегий в условиях конкуренции на рынке, разрешение политических и социальных конфликтов.

Задача об оптимальном распределении инвестиционных ресурсов в рамках долгосрочного планирования работы предприятий относится к задачам перспективного планирования и управления. Этот класс задач, как упоминалось ранее, требует формирования оптимального плана многоэтапной экономической операции, когда результаты каждого последующего этапа зависят от предыдущего. Возможность разбиения

задачи на этапы и отсутствие специальных ограничений на природу, характер и свойства функций также являются хорошим подспорьем для того, чтобы использовать для решения экономической задачи метод динамического программирования. Однако данный метод очень чувствителен к правильному подбору динамического шага, определяющему набор возможных состояний системы, выделяемому объему памяти, а также к самой формулировке задачи для его успешного применения в сравнении с аналогами.

В связи со всем вышеперечисленным целью данной выпускной квалификационной работы является автоматизация решения задачи оптимального инвестирования на основе динамического программирования.

Для реализации цели ставились и решались следующие задачи.

1. Формализация задачи оптимального распределения ресурсов для конкретной предметной области (инвестирование).
2. Проектирование и разработка программы, использующая в своей основе метод динамического программирования.
3. Исследование результатов решения в зависимости от начальных данных и сравнение эффективности работы алгоритма с существующими аналогами.

1 Теоретические основы и методология динамического программирования

1.1 Введение в метод динамического программирования

Динамическое программирование, несмотря на название метода, изначально представляло собой метод математической оптимизации, разработанный Ричардом Беллманом в 1950-х годах. «Первоначально эта область была основана, как системный анализ и инжиниринг, которая была признана IEEE» [4]. «Вклад Беллмана в динамическое программирование был увековечен в названии уравнения Беллмана, центрального результата теории динамического программирования, который переформулирует оптимизационную задачу в рекурсивной форме» [5].

Основная идея динамического программирования заключается в разбиении сложной задачи на более простые подзадачи, решение которых можно использовать для построения решения исходной задачи. «Это достигается путем использования принципа оптимальности Беллмана, который можно сформулировать следующим образом – оптимальная стратегия для подзадачи будет оптимальной, если независимо от начального состояния и решения первой подзадачи, все последующие решения остаются оптимальными относительно состояния, которое возникает после первого шага»[4]. Иными словами, «оптимальная политика обладает тем свойством, что каковы бы ни были начальное состояние и начальное решение, последующие решения должны соответствовать оптимальной политике по отношению к состоянию, вытекающему из первого решения»[6].

Ниже рассмотрено доказательство принципа оптимальности с ориентированием на статью за авторством Сутягиной.

Пусть у нас есть начальное состояние P_0 и конечное состояние P_n . Оптимальная политика — это лучший способ достижения цели P_n из начального состояния P_0 через промежуточные состояния $P_1, P_2, \dots, P_{n-1}$ [6].

Рассмотрим доказательство на примере задачи перевозки грузов. Допустим, у нас есть несколько пунктов, через которые может проходить маршрут перевозки: P_0 (начальный пункт), $P_1, P_2, \dots, P_{n-1}$ (промежуточные пункты) и P_n (конечный пункт). Оптимальная политика — это путь, минимизирующий затраты на перевозку груза от P_0 до P_n . Пусть он проходит через пункты $P_1, P_2, \dots, P_{n-1}$, то есть оптимальный путь равен $P_0, P_1, P_2, \dots, P_{n-1}, P_n$. Теперь рассмотрим альтернативный маршрут, который отличается от оптимального. Допустим, альтернативный маршрут проходит через пункты $P_2^* \dots P_{n-1}^*$.

Рассмотрим подзадачу нахождения минимального пути из P_1 в P_n : пусть альтернативный маршрут лучше оптимального. Это означает, что маршрут через пункты $P_1, P_2^* \dots P_{n-1}^*, P_n$ имеет меньшие затраты, чем маршрут через пункты $P_1, P_2, \dots, P_{n-1}, P_n$ и является оптимальным для подзадачи перехода из пункта P_1 в конечный пункт P_n .

Это означает, что подпуть $P_1, P_2^* \dots P_{n-1}^*, P_n$ также должен быть частью общего оптимального маршрута $P_0, P_1, P_2^* \dots P_{n-1}^*, P_n$. Но путь $P_0, P_1, P_2, \dots, P_{n-1}, P_n$ уже был назван оптимальным, а это означает, что не существует лучшего маршрута, чем $P_0, P_1, P_2, \dots, P_{n-1}, P_n$.

Таким образом, предположение о том, что альтернативный маршрут через пункты $P_2^* \dots P_{n-1}^*$ является лучшим, приводит к противоречию. Следовательно, наше предположение неверно, и исходный путь действительно является оптимальным. То есть, оптимальный маршрут из любого промежуточного пункта до конечного должен совпадать с частью общего оптимального маршрута.

Беря во внимание принцип оптимальности, при формировании решения методом динамического программирования учитываются следующие три основных положения.

1. Оптимальная подструктура: оптимальное решение задачи включает в себя оптимальные решения её подзадач.

2. Перекрывающиеся подзадачи: это означает, что исходная задача и ее подзадачи «представляют набор операций, который выполняется многократно для получения искомого результата». [7].

3. Мемоизация: это свойство связано с использованием памяти для сохранения решений подзадач, которые могут встретиться повторно и соответственно вычисляться не один раз. Это позволяет избежать повторного вычисления решений подзадач, что приводит к значительному ускорению работы алгоритма

Алгоритм решения задачи ДП состоит из двух этапов.

1. Обратный ход. При обратном ходе происходит поиск условно-оптимальных решений. Изначально известно: конечное состояние системы, которое необходимо достичь. Из этого состояния последовательно получаем решения к начальному состоянию. «Таким образом, при обратном ходе происходит последовательный поиск решения, начиная с самого последнего и заканчивая первым»[8].

2. Прямой ход. После обратного хода получено множество условно-оптимальных решений. «Теперь, когда известно начальное состояние системы необходимо отбросить неоптимальные решения, получив, таким образом, оптимум для всего процесса в общем»[8].

Формализация принципа оптимальности Беллмана и его использование для нахождения оптимального решения многопериодных задач достигается за счет уравнение Беллмана, формулировка которого приведена в следующем пункте

1.2 Методология решения на основе рекуррентных уравнений

Функция Беллмана имеет следующий вид:

$$B_k(s) = V^*(1)$$

Где:

V^* – оптимальное значение

k – шаг динамического программирования.

s – исходное состояние

В соцветии с формулировкой принципа оптимальности, каждый шаг и значение функции зависит от следующих составляющих:[9]

- результата $B_0(s)$, то есть от начального состояния;
- результата $B_{k-1}(s)$.

Рассмотрим формализацию задачи и рекуррентных уравнений Беллмана на примере поиска минимального пути на графе. Будем использовать следующие обозначения:

- s_0 – начальная вершина, точка старта;
- s_n – конечная вершина, точка назначения;
- s_k – текущая вершина на шаге k ;
- $x(s)$ – функция управление перехода между вершинами, то есть запись $s_k = x_k(s_{k-1})$ обозначает переход из вершины s_{k-1} в вершину s_k ;
- $l(s_{k-1}, s_k)$ – расстояние между вершинами s_{k-1} и s_k .

Пусть $\Gamma(d)$ - множество вершин, доступных из вершины d за один шаг. Тогда функция эффективности каждого шага может быть выражена как минимальное расстояние между вершинами:

$$L_k = \min_{d \in \Gamma(s_k)} [l_k(s_k, d)] (2)$$

Целевая функция будет вычисляться как сумма результатов всех функций эффективности на каждом шаге:

$$L = \sum_{k=1}^n L_k (3)$$

Нам необходимо найти кратчайший путь из s_0 в s_n определив все траектории направлений $x(s)$

Рассмотренный ранее принцип оптимальности гласит, что данную

задачу нахождения минимального пути можно свести к последовательности простых задач нахождения минимального пути между соседними вершинами.

Для этого сначала рассмотрим функцию Беллмана для $k = 0$. Очевидно, что в этом случае s_0 будет являться целевой вершиной и минимальный путь из вершины в саму себя будет равен 0, поэтому формула (1) приобретет следующий вид:

$$B_0(s_0) = 0 \quad (4)$$

На шаге $k = 1$ минимальный путь будет равен пути от s_1 до s_0 , где s_1 позволяет попасть в вершину s_0 . Согласно принципу оптимальности предпоследний шаг должен быть оптимальным, поэтому переход из s_1 в s_0 обязан быть оптимальным. Также важно учесть факт того, что у вершины s_k на k шаге оптимизации может быть множество соседних вершин, из которого нам нужно выбрать ту вершину s_{k-1} , которая обеспечит нам оптимальное решение (минимальное расстояние между s_k и s_{k-1}). Тогда для $k=1$ функция Беллмана и формула (4) принимают следующий вид в соответствии с уравнением (2):

$$\begin{cases} B_1(s_1) = \min_{s_1 \in \Gamma(s_0)} [l(s_1, s_0) + B_0(s_0)] \\ B_0(s_0) = \begin{cases} 0 & \text{— если } s_0 \text{ — целевая вершина;} \\ \infty & \text{— в противном случае.} \end{cases} \end{cases} \quad (5)$$

Учитывая формулу (5), мы можем составить общие рекуррентные уравнения Р. Беллмана для решения исходной n мерной задачи оптимизации:

$$\begin{cases} B_k(s_k) = \min_{s_{k-1} \in \Gamma(s_k)} [l(s_k, s_{k-1}) + B_{k-1}(s_{k-1})] \\ B_0(s_0) = \begin{cases} 0 & \text{— если } s_0 \text{ — целевая вершина;} \\ \infty & \text{— в противном случае.} \end{cases} \end{cases} \quad k = 1 \dots n \quad (6)$$

1.3 Практическое применение метода динамического программирования

Пример № 1. Задача о станках

Детали n видов могут обрабатываться на двух станках. Время обработки i -й детали ($i=1, n$) на первом станке равно a , минут, а время обработки той же детали на втором станке равно b , минут, «Очередность обработки деталей одна и та же: сначала деталь обрабатывается на первом станке, а затем на втором. Требуется выбрать такую последовательность обработки деталей, при которой время изготовления всех деталей является минимальным»[10].

Пример №2. Задача о рюкзаке.

Задача о рюкзаке (Knapsack Problem) является классической задачей комбинаторной оптимизации. Она формулируется следующим образом: имеется рюкзак определенной вместимости и множество предметов, каждый из которых имеет определенный вес и ценность. Необходимо выбрать такой набор предметов, чтобы максимизировать суммарную ценность предметов в рюкзаке, при этом, не превышая его вместимость. Операция, которая рассматривается в данной задаче, является многоэтапной.

Пример №3. «Задача об оптимальном режиме набора высоты и скорости летательным аппаратом»[11]

«Задача состоит в следующем. Самолет (или другой летательный аппарат). находящийся на высоте H_0 и имеющий скорость V_0 , должен быть поднят на заданную высоту $H_{\text{кон}}$, а скорость его доведена до заданного значения $V_{\text{кон}}$. Известен расход горючего, потребный для подъема аппарата с любой высоты H_1 на любую другую $H_2 > H_1$ при неизменной скорости V ; известен также расход горючего, потребный для увеличения скорости от любого значения V_1 до любого другого $V_2 > V_1$ при неизменной высоте H . Требуется найти оптимальный режим набора высоты и скорости, при котором общий расход горючего будет минимальным»[12]

Требуется найти такой режим набора высоты и скорости, при котором общий расход топлива будет минимальным.

1.4 Заключение по теоретическим основам

В первой главе выпускной квалификационной работы рассмотрены и проанализированы:

- основные аспекты и принципы динамического программирования.
- принцип оптимальности и рекуррентные уравнения Беллмана как основной механизм метода ДП
- примеры практических задач, которые можно решить методом ДП

2 Практическая часть

2.1 Формализация задачи об оптимизации распределения ресурсов

Оптимизация распределения ресурсов — это процесс определения наилучшего способа использования ограниченных ресурсов для достижения определенных целей или удовлетворения определенных потребностей. Эта проблема возникает в различных областях, таких как экономика, производство, логистика, управление проектами, исследование операций и др.

Общие положения:

- **Ограниченность ресурсов:** В большинстве случаев доступные ресурсы, такие как время, деньги, трудовые ресурсы, материалы и т.д., ограничены, что создает необходимость их оптимального распределения.
- **Цели:** Цели оптимального распределения ресурсов могут быть различными в зависимости от поставленной задачи, например максимизация прибыли, минимизация издержек, оптимизация производственных процессов, удовлетворение потребностей потребителей и т.д.
- **Ограничения:** Существуют различные ограничения, которые могут влиять на процесс распределения ресурсов, такие как технические, финансовые, временные, правовые и другие.

В связи с тем, что метод динамического программирования очень требователен к постановке задачи, а также в связи с отсутствием реальных данных или реального заказчика было принято решения взять классическую задачу о распределении инвестиционного капитала между несколькими предприятиями для увеличения прибыли от выпуска продукции.

Также был выдвинут ряд предположений для упрощения задачи, которые способствуют более наглядной демонстрации принципа работы метода динамического программирования. Такой подход также позволил включить большее количество алгоритмов-аналогов в исследовательскую

часть ВКР для наглядного сравнения эффективности работы этих алгоритмов в сравнении с методом динамического программирования.

Описание задачи оптимального распределения ресурсов:

«Необходимо распределить ограниченные средства S между N предприятиями производственного объединения»[3].

Предположения.

1. Прибыль, полученная филиалами в рамках распределения денежных средств, также формирует общую прибыль всего предприятия.
2. Получаемая предприятием прибыль зависит от размера вложенных в него средств.
3. Функция прироста прибыли предприятий и их значения в зависимости от вкладываемой суммы заранее определены и заданы дискретно в виде таблицы. (см таблицу 1)
4. Прирост прибыли в филиалах не зависят друг от друга.

«В качестве одного шага задачи динамического программирования будем рассматривать факт выделения средств одному предприятию. Всего будет N шагов. Учитывая, что функции прибыли заданы таблично, разобьем отрезок $[0; S]$ на шаги»[3]. Эти шаги разделены некоторым интервалом Δ . Всего m шагов. Значения функций, входящих в уравнения Беллмана, будут определяться только в точках, кратных Δ .

«На начальном этапе необходимо последовательно определить оптимальное распределение между предприятиями средств в объеме $x = \Delta, 2\Delta, \dots, l\Delta, \dots, S$. Анализируя оптимальное распределение каждой суммы, необходимо найти оптимальное распределение средств между предприятиями $X^* = (x_1^* \dots x_n^*)$ »[3]

Таким образом, переменными модели являются:

S – общий объем распределяемых средств.

k, l – шаг, $k = 1 \dots n, l = 1 \dots m$

$\Delta = S/m$ – интервал

$x_k \in (0, \Delta, \dots, S)$ – капиталовложение в k филиал

s_{kl} – денежный объем l варианта распределения, выделяемый k-му филиалу;

Параметры задачи имеет смысл представить в виде таблицы 1 для упрощения понимания:

Таблица 1 – Прирост продукции на предприятиях

Распределенные средства S	Вклады аемые средства x	Функции прибыли предприятия			
		$F_1(x_1)$	$F_2(x_2)$	...	$F_n(x_n)$
S	0	s_{11}	s_{21}	...	s_{n2}
	Δ	s_{12}	s_{22}	...	s_{n2}
	...	...	...	...	...
	$l\Delta$	s_{1l}	s_{2l}	...	s_{nl}
	...	...	...	...	...
	S	s_{1m}	s_{2m}	...	s_{nm}

$F_k(x_k)$ – функция прибыли при выделенных x_k средствах для k-го филиала.

$X = \sum_{k=1}^n (x_k)$ – общая сумма выделенных средств (на каждом шаге филиалу k выделяется x_k средств), $0 \leq X \leq S$;

Целевая функция:

$$F = \sum_{k=1}^n F_k(x_k) \rightarrow \max (7)$$

Ограничения:

$$\sum_{k=1}^n (x_k) = S, x_k \geq 0 (8)$$

Для реализации метода динамического программирования в решении данной задачи, необходимо добиться реализации шагов динамического

программирования, то есть разбить основную задачу поиска максимума целевой функции на подзадачи.

Пусть $Z_n(x)$ – функция, обозначающая максимальный прирост для вложенных средств x и количестве предприятий n .

Очевидно, что максимальная прибыль для одного предприятия будет равна приросту этого предприятия:

$$Z_1(x) = \max_{k=1...n} F_k(x) \quad (9)$$

Теперь рассмотрим два предприятия. Введем значение x_2 – вкладываемые средства во вторую фирму. Тогда при общей сумме x первая фирма получит $x_1 = x - x_2$.

Тогда формула принимает вид:

$$Z_2(x) = \max_{0 \leq x_2 \leq S} (F_2(x_2) + Z_1(x - x_2)), x_2 \in (0, \Delta, \dots, S) \quad (10)$$

Для трех предприятий формула преобразуется до:

$$Z_3(x) = \max_{0 \leq x_3 \leq S} (F_3(x_3) + Z_2(x - x_3)), x_3 \in (0, \Delta, \dots, S) \quad (11)$$

Несложно заметить, что для расчета суммарного максимального прироста для любого количества фирм формула формализуется в:

$$\begin{cases} Z_1(x) = \max_{k=1...n} F_k(x) \\ Z_k(x) = \max_{0 \leq x_k \leq S} (F_k(x_k) + Z_{k-1}(x - x_k)) \forall x_k \in (0, \Delta, \dots, S), k \in (1, 2, \dots, n) \end{cases} \quad (12)$$

Составив рекуррентные уравнения Беллмана, можно начинать условную оптимизацию решения применив обратный ход. Для получения максимальной прибыли необходимо проверить все $x_k \leq x$, кратные Δ (согласно постановке задачи), поэтому при решении придерживаемся следующих шагов.[13]

1. Рассчитать значения $Z_k(x) = s^*$ и $F_k(x)$ ($k=1, \dots, n$) для всех возможных x согласно (10), также запоминая значение $x^* = x_k$ для каждого рассчитанного $Z_k(x)$ и $F_k(x)$ при 1 варианте распределения (для $Z_1(x)$ значение $x^* = x$). В результате получим таблицу 2 и таблицу 3:

Таблица 2 – Максимальная прибыль

Распределяемые средства S	Вкладываемые средства x	Сумма максимальной прибыли			
		$Z_1(x_1)$	$Z_2(x_2)$	...	$Z_n(x_n)$
S	0	s_{11}^*	s_{21}^*	...	s_{n1}^*
	Δ	s_{12}^*	s_{22}^*	...	s_{n2}^*
	...	...	...	...	...
	$l\Delta$	s_{1l}^*	s_{2l}^*	...	s_{nl}^*
	...	...	...	...	...
	S	s_{1m}^*	s_{2m}^*	...	s_{nm}^*

Таблица 3 – Оптимальные управления

Вкладываемые средства x	Оптимальные управления			
	$Z_1(x)$	$Z_2(x)$	...	$Z_n(x)$
0	x_{11}^*	x_{21}^*	...	x_{n1}^*
Δ	x_{12}^*	x_{22}^*	...	x_{n2}^*
...	...	...	...	...
$l\Delta$	x_{1l}^*	x_{2l}^*	...	x_{nl}^*
...	...	...	...	...
S	x_{1m}^*	x_{2m}^*	...	x_{nm}^*

2. Находим $Z_n(S)=F^*$ являющийся максимальным значением целевой функции.

После этого, начинается этап безусловной оптимизации или прямой ход, разделенный на следующие шаги.

1. Зная максимальное значение целевой функции $Z_n(S)=F^*$ и значение $x_n = x^*$ из таблиц 2 и 3 при помощи формулы 10 находим объем финансирования оставшихся n-1 филиалов и добавляем x^* в X^* :

$$Z_{n-1}(S - x_n) = Z_n(S) - F_n(x_n) \quad (13)$$

2. Аналогично находим объемы финансирования оставшихся

филиалов, пока не достигнем $Z_1(x)$ и не заполним X^* оптимальными управлениями

2.2 Пример оптимизации распределения ресурсов на практике

Компания АвтоВаз в связи с повышением прибыли прошлого квартала приняла решение выделить в текущем квартале бюджет в размере $S = 400$ млн. р. Который необходимо распределить между четырьмя самыми прибыльными филиалами-предприятиями. Для упрощения прогнозирования было принято решение выделять средства в размерах кратных $\Delta = 80$ млн. р. Вырученная прибыль от продажи продукции в зависимости от вложенных средств в каждое предприятие приведен в таблице 4. Требуется распределить вложения между филиалами так, чтобы общий прирост прибыли (в млн. р.) был максимальным.

Таблица 4 – Прирост продукции на предприятиях

Распределяемые средства S	Вкладываемые средства x	Функции прибыли предприятия			
		$F_1(x_1)$	$F_2(x_2)$	$F_3(x_3)$	$F_4(x_4)$
400	0	10	15	13	14
	80	13	20	17	16
	160	16	22	21	23
	240	21	25	26	25
	320	25	30	28	27
	400	25	32	30	32

Ход решения.

Обратный ход (условная оптимизация).

1. Рассчитать значения $Z_k(x)$ и $F_k(x) = s^*$ ($k=1, \dots, n$) для всех возможных x согласно формуле (12), запоминая значение $x^* = x_k$ для каждого рассчитанного $Z_k(x)$ и $F_k(x)$:

$$Z_1(0) = 10, x_{11}^* = 0$$

$$Z_1(80) = 13, x_{12}^* = 80$$

$$Z_1(160) = 16, x_{13}^* = 160$$

$$Z_1(240) = 21, x_{14}^* = 240$$

$$Z_1(320) = 25, x_{15}^* = 320$$

$$Z_1(400) = 25, x_{16}^* = 400$$

$$Z_2(0) = \max(10 + 15) = 25, x_{21}^* = 0$$

$$Z_2(80) = \max(13 + 15, 10 + 20) = 30, x_{22}^* = 80$$

$$Z_2(160) = \max(16 + 15, 13 + 20, 10 + 22) = 33, x_{23}^* = 80$$

$$Z_2(240) = \max(10 + 25, 13 + 22, 16 + 20, 21 + 15) = 36, x_{24}^* = 0$$

$$Z_2(320) = \max(10 + 30, 13 + 25, 16 + 22, 21 + 20, 25 + 15) = 41, x_{25}^* = 80$$

$$Z_2(400) = \max(10 + 32, 13 + 30, 16 + 25, 21 + 22, 25 + 20, 25 + 15) = 45, x_{26}^* = 80$$

$$Z_3(0) = \max(13 + 25) = 38, x_{31}^* = 0$$

$$Z_3(80) = \max(13 + 30, 17 + 25) = 43, x_{32}^* = 0$$

$$Z_3(160) = \max(13 + 33, 17 + 30, 21 + 25) = 47, x_{33}^* = 80$$

$$Z_3(240) = \max(13 + 36, 17 + 33, 21 + 30, 26 + 25) = 51, x_{34}^* = 160$$

$$Z_3(320) = \max(13 + 41, 17 + 36, 21 + 33, 26 + 30, 28 + 25) = 56, x_{35}^* = 240$$

$$Z_3(400) = \max(13 + 45, 17 + 41, 21 + 36, 26 + 33, 28 + 30, 30 + 25) = 59, x_{36}^* = 240$$

$$Z_4(0) = \max(14 + 38) = 52, x_{41}^* = 0$$

$$Z_4(80) = \max(14 + 43, 16 + 38) = 57, x_{42}^* = 0$$

$$Z_4(160) = \max(14 + 47, 16 + 43, 23 + 38) = 61, x_{43}^* = 0$$

$$Z_4(240) = \max(14 + 51, 16 + 47, 23 + 43, 25 + 38) = 66, x_{44}^* = 160$$

$$Z_4(320) = \max(14 + 56, 16 + 51, 23 + 47, 25 + 43, 27 + 38) = 70, x_{45}^* = 0$$

$$Z_4(400) = \max(14 + 59, 16 + 56, 23 + 51, 25 + 47, 27 + 43, 32 + 38) = 74, x_{46}^* = 160$$

Результаты расчетов отражены в таблице 5 и таблице 6.

Таблица 5 – Рассчитанная максимальная прибыль

Распределяемые средства S	Вкладываемые средства x	Сумма максимальной прибыли			
		$Z_1(x)$	$Z_2(x)$	$Z_3(x)$	$Z_4(x)$
400	0	10	25	38	52
	80	13	30	43	57
	160	16	33	47	61
	240	21	36	51	66
	320	25	41	56	70
	400	25	45	59	74

Таблица 6. – Рассчитанные оптимальные управления

Вкладываемые средства x	Оптимальные управления			
	$Z_1(x)$	$Z_2(x)$	$Z_3(x)$	$Z_4(x)$
0	0	0	0	0
80	80	80	0	0
160	160	80	80	0
240	240	0	160	160
320	320	80	240	0
400	400	80	240	160

2. $Z_4(400) = 74$

Прямой ход (безусловная оптимизация).

Найдем, какие значения дают наибольшее значение для $Z_4(400) = 23 + 51 = 74$, то есть $F_4(160) + Z_3(240)$. Тогда в четвертое предприятие нужно вложить 160 млн. р. Аналогично для $F_3(240) = 26 + 25$, то есть $F_3(160) + Z_2(80)$. В третье предприятие нужно вложить 160 млн. р. Для $F_2(80) = 10 + 20$, то есть $F_2(80) + Z_1(0)$. Тогда во второе месторождение нужно вложить 80 млн. р., а в первое не вкладывать ничего.

Финальный ответ представлен в таблице 7.

Таблица 7. – Оптимальное распределение

Филиал	Ф1	Ф2	Ф3	Ф4
Итоговое распределение	0	80	160	160

2.3 Структурная схема задачи об оптимизации распределения ресурсов.

Структурная схема – это графическое представление структуры предметной области, системы, объектов, организации или задачи, которое показывает элементарные звенья и связь между ними. Она необходима для анализа и возможности конструировать и рассчитывать сложные системы и объекты, а также для того, чтобы понимать их принципы действия и функции.

На рисунке 1 приведена структурная схема, описывающая общий алгоритм решения задачи оптимального распределения ресурсов методом динамического программирования.

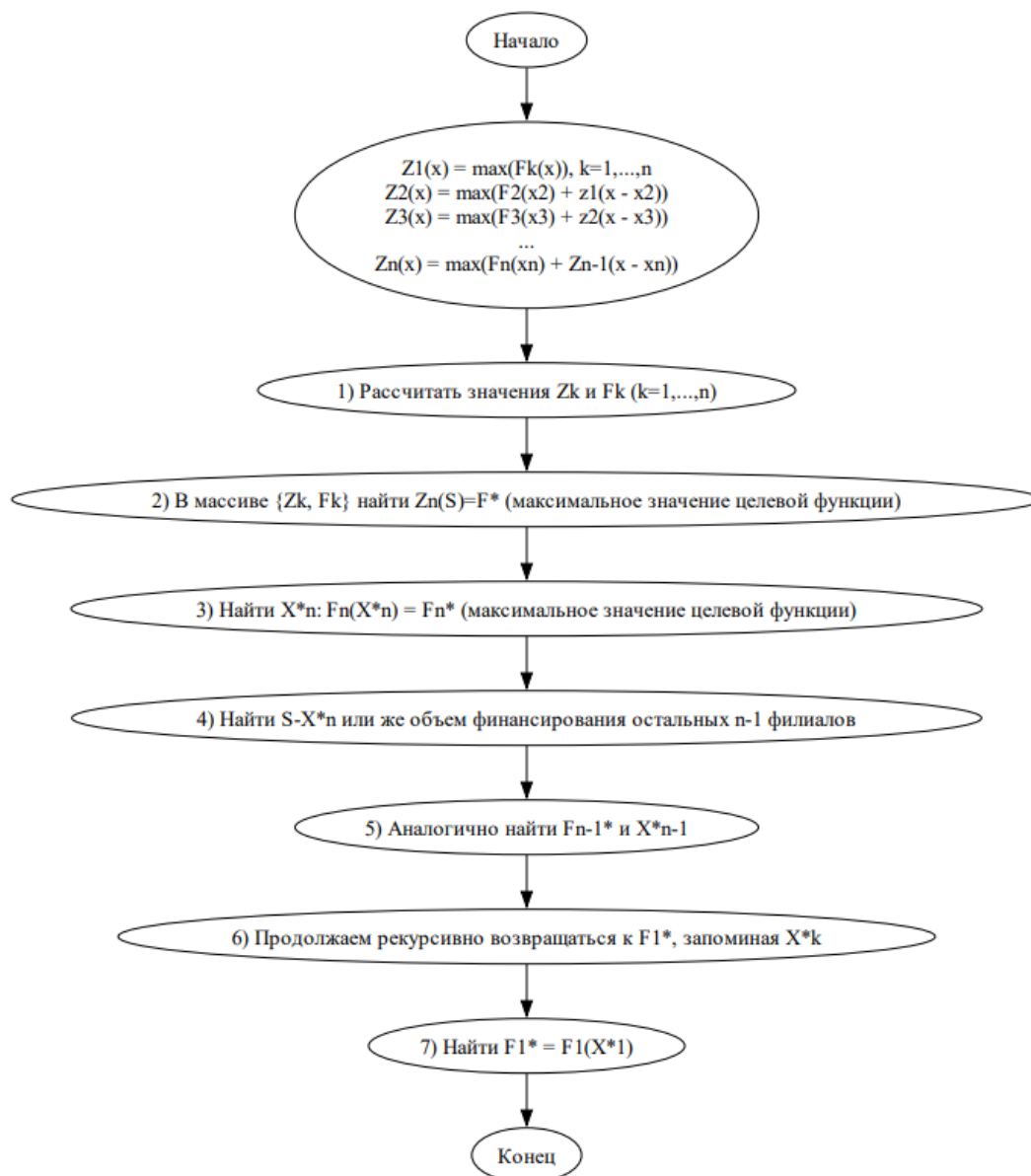


Рисунок 1 – Структурная схема

2.4 Заключение по практической части

В данной главе была сформулирована постановка задачи оптимального распределения ресурсов и представлена ее математическая модель.

Представлено аналитическое решение задачи оптимального распределения ресурсов с применением метода динамического программирования.

3 Программное решение задачи оптимизации распределения ресурсов на основе метода динамического программирования

3.1 Подход к реализации ПО

Для разработки ПО, обеспечивающего решение задачи оптимального распределения ресурсов, было принято решение использовать стратегию минимально жизнеспособного продукта.

«Минимально жизнеспособный продукт (англ. minimum viable product, MVP) — продукт, обладающий минимальными, но достаточными для удовлетворения первых потребителей функциями.»[14] «Основная задача MVP — получение обратной связи для формирования гипотез и дальнейшего развития продукта.»[14]

Основные принципы при разработке MVP.

1. Фокус на ключевые функции.
2. Быстрое развертывание.
3. Минимальные затраты.

В рамках данной квалификационной работы стратегия разработки MVP была выбрана по следующим причинам.

1. Исследовательский аспект.

Разработка MVP позволит изучить и продемонстрировать возможности применения метода динамического программирования на практике. Даже без реальной задачи от заказчика, создание MVP с тестовыми данными и типовыми задачами динамического программирования, формализованными в предыдущих главах, будет полезно для исследовательских и обучающих целей.

2. Отсутствие реальных данных.

При отсутствии реальных данных и задачи от заказчика, MVP с использованием учебных данных позволит быстро протестировать функциональность алгоритма. Это поможет понять, насколько динамическое

программирование эффективно и применимо для решения рабочих задач оптимизации.

3. Минимальные затраты.

Разработка MVP потребует меньше ресурсов, чем разработка полноценного приложения. Это позволяет быстро протестировать идею и в дальнейшем, при наличии реальных данных и задачи от заказчика, расширить MVP в соответствии с их требованиями.

В соответствии с вышеперечисленными положениями была разработана следующая USE-CASE диаграмма (см. рисунок 2), отражающая стратегию, в соответствии с которой будет проводиться дальнейшее разработка ПО:

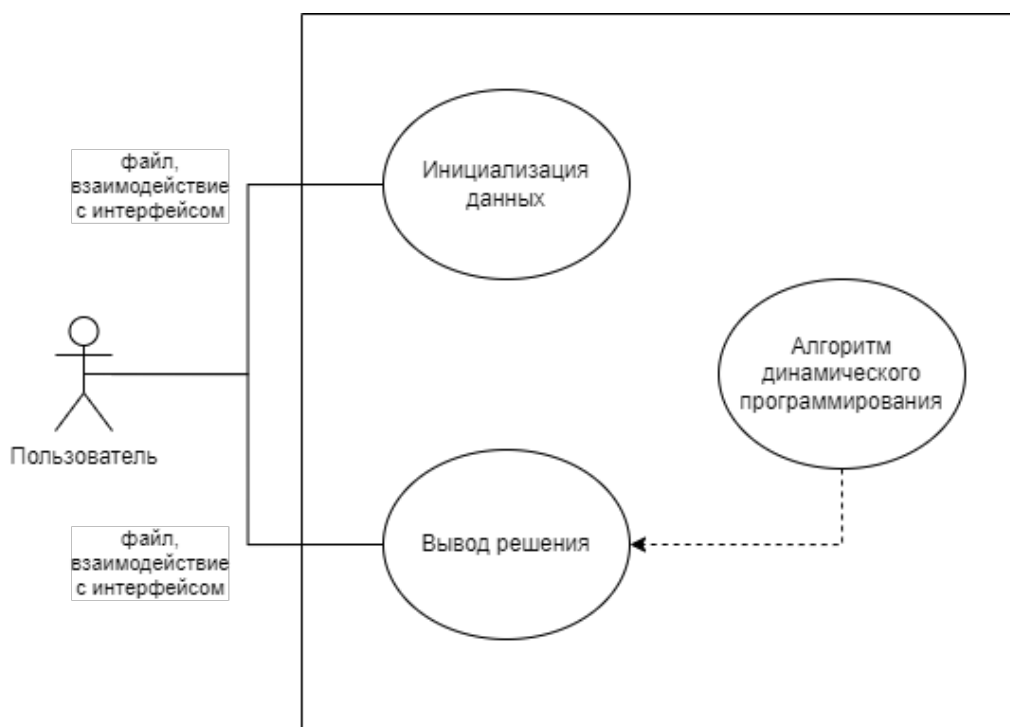


Рисунок 2 – Use Case диаграмма

3.2 Стек технологий ПО

При разработке ПО рассматривались такие языки программирования, которые имеют набор API или готовых библиотек для реализации простого интерфейса с CRUD логикой, а также библиотеки с готовой реализацией

эвристических алгоритмов, алгоритмов целочисленного программирования.

Python

Python – это интерпретируемый высокоуровневый язык программирования с динамической типизацией и автоматическим управлением памятью. Он поддерживает несколько парадигм программирования, включая структурное, объектно-ориентированное и функциональное программирование.[15]

Библиотеки.

- GUI: Tkinter, PyQt, wxPython.
- Эвристические алгоритмы: DEAP (Distributed Evolutionary Algorithms in Python).
- Алгоритмы целочисленного программирования: PuLP, OR-Tools (Google Optimization Tools).

Java

Java – это объектноориентированный языкпрограммирования общего назначения, который специально разработан для того, чтобы иметь как можно меньше зависимостей при реализации.

Библиотеки.

- GUI: Swing, JavaFX.
- Эвристические алгоритмы: JGAP (Java Genetic Algorithms Package).
- Алгоритмы целочисленного программирования: Apache Commons Math, OR-Tools.

C#

C# – это объектно-ориентированный язык программирования, разработанный компанией Microsoft в рамках платформы .NET. Он предназначен для разработки разнообразных приложений, включая настольные приложения, веб-сервисы и мобильные приложения.

Библиотеки.

- GUI: Windows Forms, WPF (Windows Presentation Foundation).
- Эвристические алгоритмы: Accord.NET.
- Алгоритмы целочисленного программирования: Google OR-Tools.

C++

C++ – это компилируемый язык программирования общего назначения, который поддерживает как процедурное, так и объектно-ориентированное программирование, а также некоторые элементы функционального программирования.

Библиотеки.

- GUI: Qt, wxWidgets.
- Эвристические алгоритмы: Evolving Objects (EO), Shark.
- Алгоритмы целочисленного программирования: COIN-OR (Computational Infrastructure for Operations Research), Google OR-Tools.

JavaScript (с использованием Electron)

JavaScript — это язык программирования, обычно используемый для разработки вебприложений, но также применяемый и в других средах благодаря таким платформам, как Node.js.

Библиотеки.

- GUI: Electron, React.js (с использованием Electron для настольных приложений).
- Эвристические алгоритмы: genetic.js.
- Алгоритмы целочисленного программирования: glpk.js (JavaScript binding for GLPK).

При выборе языка программирования рассматривались достоинства и недостатки языков, представленных в таблице 7.

Таблица 8 – Плюсы и минусы языка программирования для решения задачи динамического программирования

Языки программирования	Плюсы	Минусы
Python	1. Удобный и понятный синтаксис. 2. Кроссплатформенность: работает на всех основных операционных системах. 3. Огромное количество библиотек и модулей. 4. Интерпретируемый язык. 5. Быстрая разработка по сравнению с другими языками. 6. Имеется огромное количество ресурсов для обучения, большинство документации переведено на русский язык. 7. Бесплатный, не требует лицензии.	1. Нестабильность и несовместимость библиотек из-за большого количества авторов. 2. Медленная работа по сравнению с компилируемыми языками. 3. Требуется много памяти для больших проектов.
Java	1. Кроссплатформенность благодаря JVM. 2. Интерпретируемый язык. 3. Объектно-ориентированный подход. 4. Много встроенных библиотек и функций. 5. Высокая безопасность и надежность. 6. Много ресурсов для обучения и документации на русском языке. 7. Бесплатный.	1. Медленнее по сравнению с компилируемыми языками. 2. Нестабильность и несовместимость версий из-за большого количества обновлений. 3. Многословный синтаксис, усложняющий изучение и использование.
C#	1. Высокая скорость работы благодаря компиляции в машинный код. 2. Универсальность и	1. Сложный синтаксис для изучения. 2. Нестабильность и несовместимость

	гибкость. 3. Хорошо поддерживается в среде Windows. 4. Много ресурсов для обучения и документации на русском языке. 5. Бесплатный (с .NET Core).	версий. 3. Много библиотеки, но менее разнообразны, чем в Python.
C++	1. Высокая скорость работы, за счет преобразования кода в машинный код. 2. Интерпретируемый язык. 3. Универсальный и гибкий. Данный язык программирования можно использовать для различных задач. 4. Много ресурсов для обучения и документации на русском языке. 5. Не требует платы за использование.	1. Сложный синтаксис для изучения и использования. 2. Требуется управление памятью вручную, что увеличивает риск ошибок. 3. Плохая поддержка модульности. 4. Необходимость компиляции. 5. Примитивный предпроцессор от языка программирования C.
JavaScript	1. Использование веб-технологий (HTML, CSS, JavaScript) для разработки интерфейсов. 2. Кроссплатформенность через Electron. 3. Быстрое прототипирование и разработка. 4. Много ресурсов для обучения и документации на русском языке. 5. Бесплатный.	1. Высокое потребление ресурсов (памяти и процессора). 2. Безопасность: уязвимости JavaScript. 3. Низкая производительность по сравнению с компилируемыми языками.

Учитывая приведенную выше информацию о языках программирования и их достоинствах и недостатках, для разработки ПО в

рамках MVP концепции был выбран Python, так как он имеет множество преимуществ по отношению к другим языкам программирования.

1. Python имеет очень простой и понятный синтаксис, который позволяет разработчикам быстро писать и читать код. Это особенно важно на начальных этапах разработки, когда требуется быстро реализовать основной функционал приложения.

2. Python обладает обширной экосистемой библиотек, которые могут значительно ускорить разработку. Для создания GUI доступны такие библиотеки, как Tkinter и PyQt, для обработки данных - pandas, а также готовые реализации алгоритмов, в виде DEAP и PuLP. Это позволяет сосредоточиться на бизнес-логике приложения, используя готовые решения для стандартных задач.

3. Является интерпретируемым языком, что позволяет выполнять код без предварительной компиляции. Это делает данный язык программирования динамически типизированным и платформонезависимым.

4. Python поддерживает все основные операционные системы (Windows, macOS, Linux). Это позволяет созданным приложениям работать на любых устройствах без значительных изменений в коде.

5. Python имеет огромное сообщество, которое постоянно развивается и участвует в создании множества учебных материалов и документации на различных языках, включая русский. Это облегчает процесс обучения и решения возникающих проблем.

6. Бесплатный язык программирования с открытым исходным кодом.

Для разработки кода на Python существует два вида ПО – редактор кода и IDE. Они имеют различные возможности и характеристики, которые рассмотрены ниже, также в рамках сравнения:

Редакторы кода

1. Microsoft Visual Studio Code (VS Code)

Преимущества:

- **Расширяемость:** огромное количество расширений для Python, включая поддержку для отладки, управления виртуальными окружениями, автодополнения кода и интеграции с Git.
- **Легкость и быстрота:** быстро загружается и работает даже на менее мощных машинах.
- **Интуитивный интерфейс:** простой и настраиваемый интерфейс, который легко освоить.
- **Интеграция с Jupyter Notebook:** возможность работы с Jupyter Notebook прямо в редакторе, что полезно для аналитики данных.
- **Кросс-платформенность:** доступен для Windows, macOS и Linux.

Недостатки:

- **Не все возможности IDE:** хотя VS Code обладает множеством возможностей, он всё же может уступать полнофункциональным IDE в некоторых аспектах.

2. Sublime Text

Преимущества:

- **Скорость:** очень быстрый и легковесный.
- **Расширяемость:** поддержка большого количества плагинов через Package Control.

- **Кросс-платформенность:** работает на Windows, macOS и Linux.

Недостатки:

- **Платный:** полная версия платная, хотя бесплатная версия доступна с некоторыми ограничениями.
- **Меньше встроенных возможностей:** требует установки плагинов для достижения функциональности, схожей с VS Code.

3. Atom

Преимущества:

- **Расширяемость:** высокая степень настраиваемости и большое

количество доступных плагинов.

- Интеграция с Git и GitHub: отличная интеграция с Git и GitHub.
- Кросс-платформенность: доступен для Windows, macOS и Linux.

Недостатки:

- Производительность: может быть медленным при работе с большими проектами.
- Разработка приостановлена: официальная поддержка прекратилась в 2023 году.

Интегрированные среды разработки (IDE)

1. PyCharm (от JetBrains)

Преимущества.

- Мощный набор функций: включает в себя автодополнение кода, отладку, тестирование, рефакторинг, поддержку Django и Flask.
- Поддержка научных библиотек: отличная поддержка для работы с библиотеками для анализа данных и машинного обучения (например, Pandas, NumPy, TensorFlow).
- Интеграция с Jupyter Notebook.

Недостатки.

- Ресурсоемкость: может быть медленным на старых или менее мощных компьютерах.
- Стоимость: Профессиональная версия платная (есть бесплатная Community Edition с ограниченными возможностями).

2. Spyder

Преимущества.

- Нацеленность на науку о данных: встроенная поддержка для научных вычислений и анализа данных.
- Интеграция с IPython и Jupyter.
- Легкость использования: интуитивный интерфейс, ориентированный на ученых и инженеров.

Недостатки.

- Ограниченные возможности для веб-разработки: менее подходит для разработки веб-приложений по сравнению с PyCharm или VS Code.

3. Jupyter Notebook

Преимущества.

- Итеративное программирование: идеально подходит для исследования данных и создания интерактивных отчетов.
- Поддержка множества языков: хотя первоначально создан для Python, поддерживает и другие языки через ядра (kernels).

Недостатки:

- Ограниченные возможности разработки: Не подходит для создания полноценных приложений.

В итоге выбор пал на VS Code. Для разработки простого оконного приложения, Visual Studio Code был выбран по следующим причинам.

1. Быстрая настройка и простота использования: VS Code легко установить и настроить, что важно для быстрого начала разработки. Он предоставляет удобный и интуитивно понятный интерфейс.

2. Расширяемость: существует множество расширений для Python, включая поддержку для отладки, автодополнения кода и управления виртуальными окружениями. Это делает разработку более эффективной и быстрой.

3. Кросс-платформенность: VS Code работает на всех основных операционных системах (Windows, macOS, Linux), что позволяет использовать его на любом рабочем компьютере.

4. Интеграция с Git: встроенная поддержка систем контроля версий (Git) облегчает управление кодом и совместную работу над проектом.

5. Поддержка Jupyter Notebook: возможность работать с Jupyter Notebook прямо в VS Code полезна для аналитики данных и экспериментов с алгоритмами.

6. Сообщество и документация: большое сообщество пользователей и обширная документация помогают быстро находить решения для возникающих проблем и улучшать навыки работы с VS Code.

Все эти факторы делают Visual Studio Code отличным выбором для быстрой разработки простого оконного приложения, предоставляя необходимые инструменты для разработки. Перед началом разработки программы необходимо было установить расширения в редакторе кода VS Code для языка программирования **Python** (см. рисунок 3).

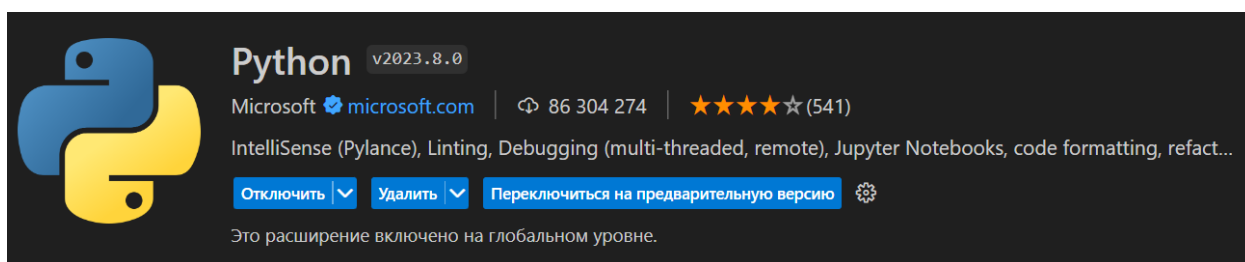


Рисунок 3 – Расширения для языка программирования Python

Для разработки интерфейса на основе языка программирования Python были рассмотрены следующие библиотеки:

Tkinter

Преимущества.

- Встроенная в Python: Tkinter является стандартной библиотекой, поэтому она поставляется с Python, и не требует дополнительной установки
- Простота: Tkinter легко изучить и использовать для создания простых GUI-приложений.
- Документация: Tkinter хорошо документирован и имеет множество примеров и ресурсов для обучения.

Недостатки.

- Ограниченные возможности: подходит для простых приложений, но может быть недостаточно мощной для сложных GUI с большим количеством элементов.

- Менее современный вид: интерфейсы, созданные с помощью Tkinter, могут выглядеть устаревшими по сравнению с современными библиотеками.

PyQt

Преимущества.

- Мощность и гибкость: PyQt основана на Qt, мощном фреймворке для создания современных GUI, что позволяет создавать сложные и профессиональные приложения.

- Большое количество виджетов: PyQt предлагает множество встроенных виджетов и поддерживает кастомизацию.

- Кросс-платформенность: приложения, написанные на PyQt, выглядят и работают одинаково на разных платформах.

Недостатки.

- Лицензирование: PyQt имеет двойное лицензирование (GPL и коммерческая лицензия). Если проект не является открытым, может понадобиться коммерческая лицензия.

- Крутая кривая обучения: из-за своей мощности и гибкости, PyQt может быть сложнее в изучении по сравнению с Tkinter.

PySide

Преимущества.

- Открытая лицензия: PySide (официально известная как Qt for Python) имеет лицензию LGPL, что делает её более привлекательной для коммерческих проектов по сравнению с PyQt.

- Совместимость с Qt: PySide предлагает аналогичные возможности и API, что и PyQt, благодаря чему можно легко переключаться между этими библиотеками.

Недостатки.

- Менее зрелая документация: несмотря на схожесть с PyQt, документация и ресурсы могут быть менее обширными.

- Поддержка и сообщество: PySide имеет меньше пользователей и ресурсов по сравнению с PyQt.

wxPython

Преимущества.

- Нативный вид: wxPython использует нативные элементы управления операционной системы, что обеспечивает нативный вид и поведение приложений.

- Широкие возможности: обширный набор виджетов и поддержка различных платформ.

Недостатки.

- Размер и сложность: wxPython это более крупная библиотека, которая может быть сложнее в использовании по сравнению с Tkinter.

- Документация и сообщество: меньше ресурсов и примеров, чем у Tkinter или PyQt.

Сравнение библиотек при разработке простого приложения.

- Tkinter: лучший выбор для быстрой разработки простого приложения из-за его простоты и интеграции с Python. Хорошо подходит для прототипирования и небольших проектов.

- PyQt / PySide: хорошие варианты для более сложных приложений, где требуются мощные возможности и современные интерфейсы. Однако, они имеют более высокую кривую обучения и требуют больше времени на разработку.

- wxPython: подходит для приложений, требующих нативного вида на всех платформах. Немного сложнее в изучении и использовании по сравнению с Tkinter, но предоставляет больше возможностей.

В связи с вышеперечисленными положениями, было принято решение остановиться на библиотеке **Tkinter**, поскольку она наиболее всего удовлетворяет философии разработки продукта в рамках концепции MVP.

Библиотека Tkinter – это стандартная библиотека Python для создания

графических интерфейсов (GUI). Она обеспечивает доступ к Tk GUI toolkit и позволяет создавать окна, диалоги, меню и другие элементы интерфейса пользователя. Ниже перечислены основные возможности библиотеки Tkinter, а также принципы и концепции построения приложения[16].

1. Инициализация основного окна в Tkinter: Первым шагом в использовании Tkinter является создание главного окна приложения при помощи функции `Tk()` (`root = tk.Tk()`)

2. Зацикливание программы: Tkinter отслеживает ввод данных пользователя, изменения в коде и обновляет интерфейс только во время активного цикла обработки событий. Делается это при помощи команды `mainloop()` по отношению к главному объекту окна `root`

3. Виджеты в Tkinter: Интерфейс пользователя в Tkinter строится при помощи использования отдельных элементов, называемых виджетами. Каждый виджет создается как объект Python из определенных библиотекой Tkinter классов: `tk.Frame`, `tk.Label` и `tk.Button` и т.д. Эти классы определяют различные элементы интерфейса, такие как текстовые метки, кнопки, текстовые поля ввода и панели.

- Структура виджетов: Виджеты в Tkinter организованы в иерархическом порядке. Например, метка и кнопка могут находиться внутри фрейма, который, в свою очередь, находится в главном окне. При создании дочернего виджета его родительский виджет передается в качестве первого аргумента конструктора виджета. Такая организация позволяет структурировать интерфейс и логически разделять различные его части.

- Настройка виджетов: У каждого виджета есть параметры конфигурации, которые определяют его внешний вид и поведение. Эти параметры можно задавать при создании виджета или изменять позже с помощью метода `config`. Например, параметры конфигурации для метки могут включать текст, цвет фона и шрифт, а для кнопки — текст и команда, выполняемая при нажатии. Различные классы виджетов имеют разные

наборы параметров

4. Управление геометрией приложения: Виджеты не появляются в интерфейсе пользователя автоматически после их создания. Для управления их расположением используется менеджер геометрии. Он определяет, где именно будут расположены и отрисованы виджеты в интерфейсе пользователя. Tkinter предоставляет три механизма для управления геометрией.

- *Place* — менеджер использует абсолютные координаты пикселя.
- *Pack* — менеджер размещает виджеты в одну из четырех сторон.

Новые виджеты размещаются рядом с существующими виджетами.

- *Grid* — менеджер размещает виджеты в сетку, похожую на динамически изменяемую таблицу.

Для хранения и загрузки данных было принято решения использовать библиотеку JSON. JSON – формат текстовых данных, который удобен для обработки данных за счет их хранения в формате пары «ключ:значение» и упорядоченных списков[17].

3.3 Блок – схема алгоритма реализации ДП

Реализуемый алгоритм решения задачи оптимального распределения ресурсов можно разбить на несколько этапов:

1. Чтение файла с исходными данными либо заполнение таблицы в интерфейсе руками пользователя
2. Выполнение обратного и прямого хода в соответствии с уравнениями Беллмана для нахождения оптимального управления и максимальной прибыли.
3. Сохранение ответа в файл или вывод результата работы алгоритма в окне интерфейса

Второй этап реализуемого алгоритма представлен на рисунке 4:

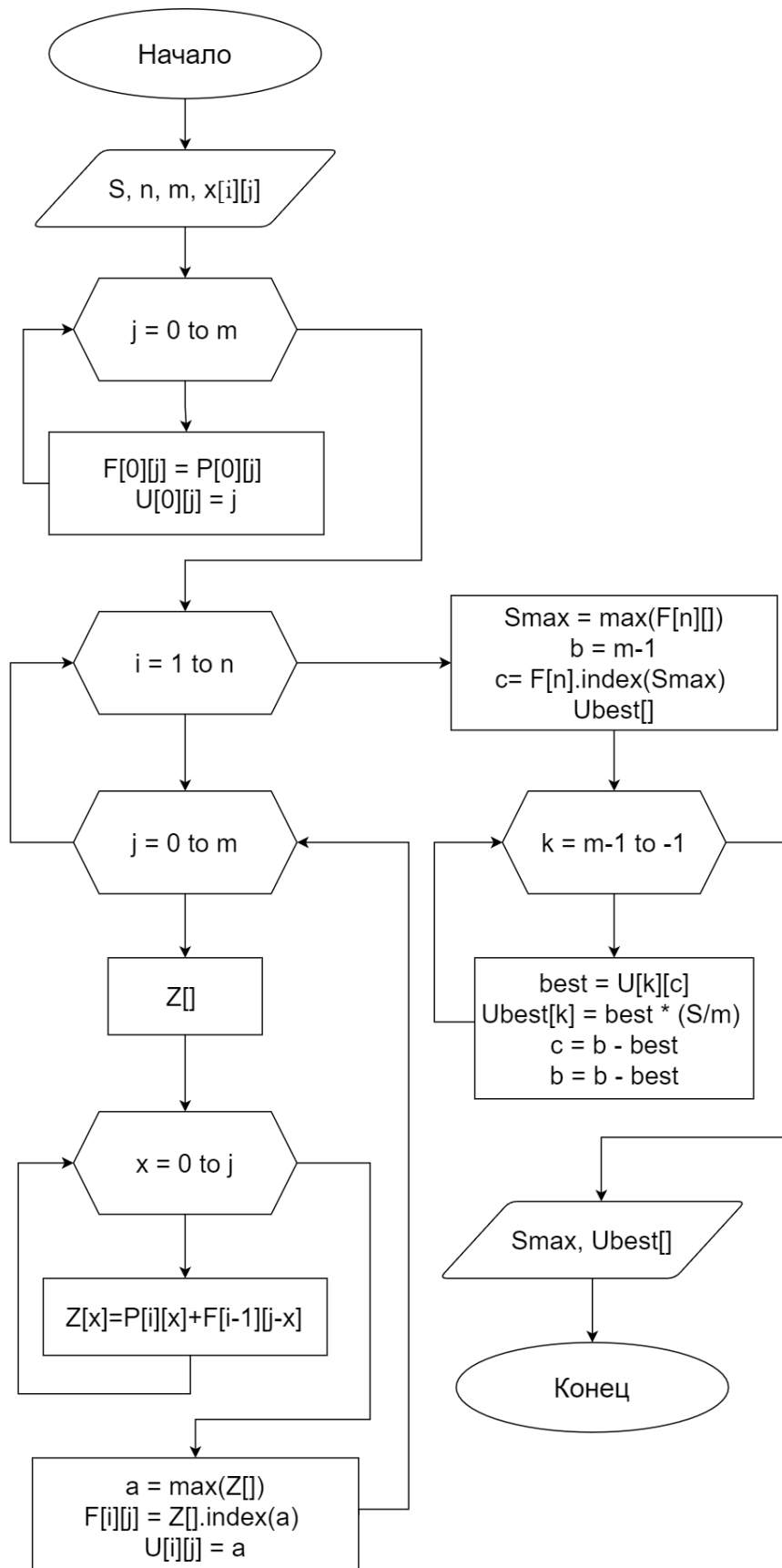


Рисунок 4 – Алгоритм динамического программирования

Где:

1. S – количество распределяемых средств).
2. Массив P размерностью $[n][m]$ где:
 - количество вариантов эффективного вложения m ;
 - количество объектов распределения (предприятий) n ;
 - критерии эффективности для каждого предприятия относительно вложения $x[i][j]$, $i = 0 \dots n-1$, $j = 0 \dots m-1$.
3. $F[n][m]$ – матрица максимальной прибыли.
4. $U[n][m]$ – матрица оптимальных переходов.
5. $S_{\max}$ – максимальная прибыль.
6. $U_{\text{best}}[n]$ – матрица оптимальных управлений.

3.4 Тестирование

Рассмотрим функционал приложения и основные варианты использования, с которыми может столкнуться пользователь. В соответствии с УС диаграммой, приведенной ранее, в данной MVP реализации пользователь способен только загрузить файл, либо ввести исходные данные при помощи элементов интерфейса, и при запуске алгоритма пользователь получает результат работы алгоритма, отраженный на элементах интерфейса, либо при желании может получить выходной файл.

На данном этапе разработки присутствуют стандартные проверки на неверный ввод, и реализован минимально необходимый интерфейс с CRUD логикой для интуитивного использования обычным пользователем. Однако, в рамках специфических сценариев использования предполагается добросовестность пользователя в связи с MVP реализацией.

Главное меню приложения содержит следующие элементы:

- Кнопка «Оптимальное распределение ресурсов»: запускает окно

где реализована основная логика программы для работы обычного пользователя

- Кнопка «Сравнение алгоритмов»: предназначена для разработчиков и запускает тесты различных алгоритмов вместе с методом динамического программирования с заранее программнозаданными параметрами и последующим выводом графиков результатов тестирования (см рисунок 5).

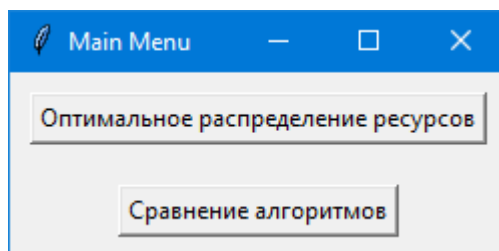


Рисунок 5 – Главное меню приложения

При нажатии кнопки «Оптимальное распределение ресурсов», пользователь увидит перед собой следующее окно (см рисунок 6).

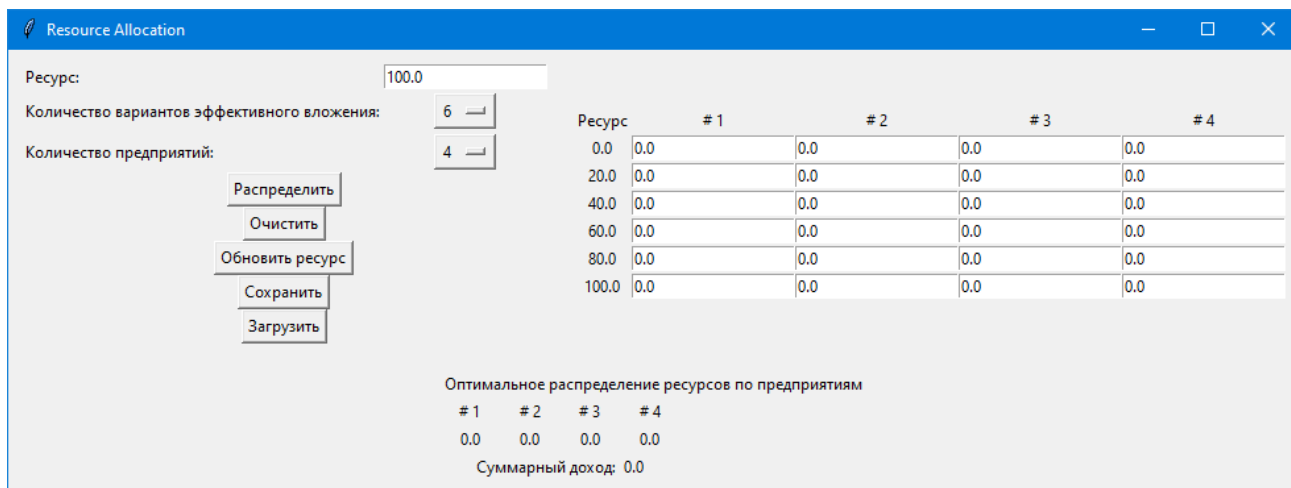


Рисунок 6 – Интерфейс приложения

На следующих рисунках приведена демонстрация возможностей, которые имеет пользователь при взаимодействии с интерфейсом, реализующих простейшую CRUD логику для взаимодействия с таблицей и ее содержимым (см. рисунки 7–15)

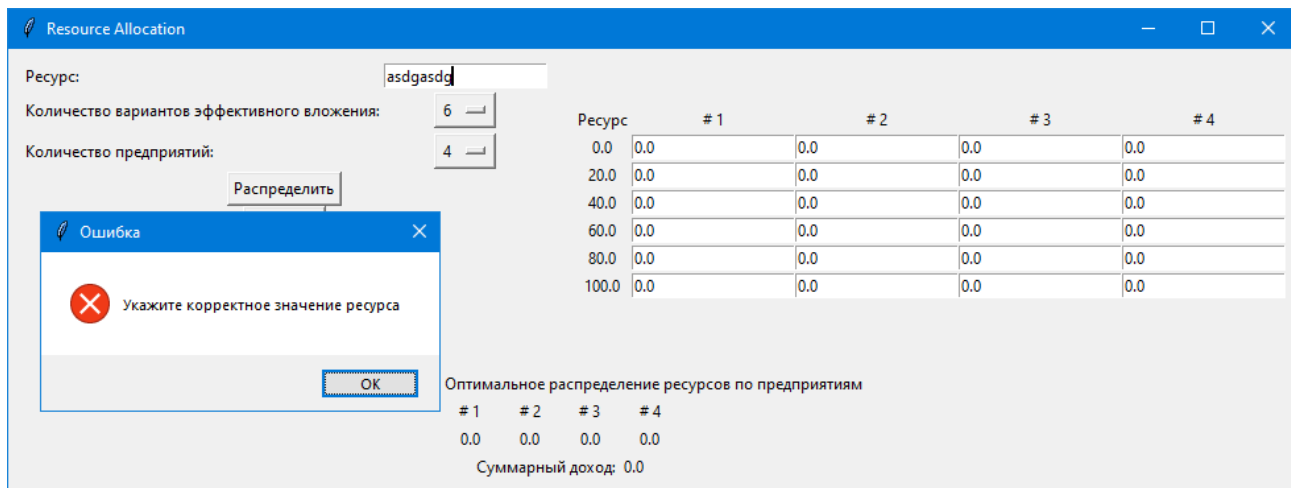


Рисунок 7 – Ввод некорректных данных ресурса

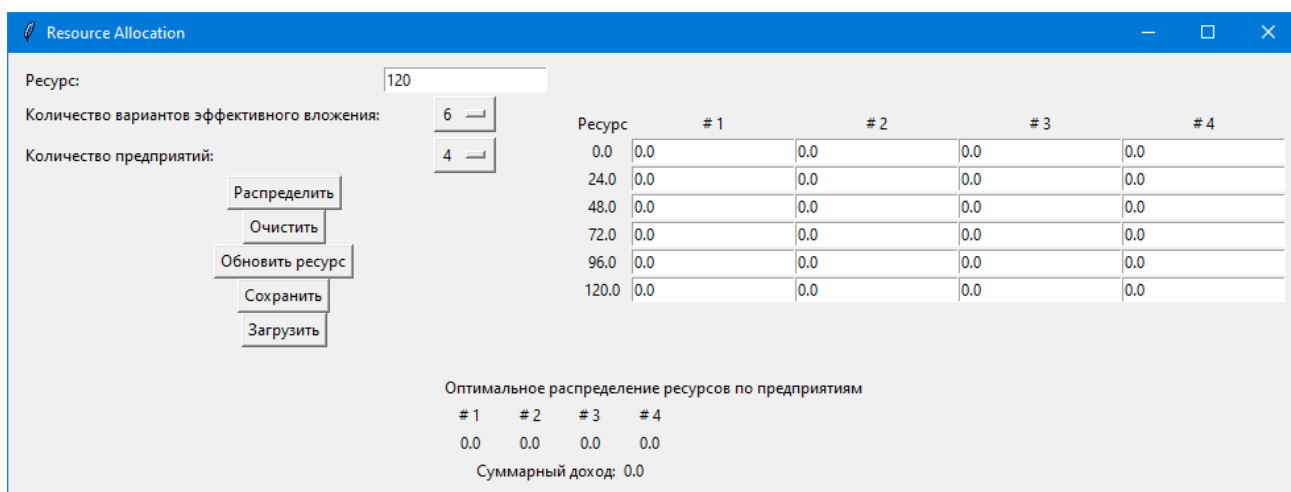


Рисунок 8 – Обновление таблицы с новым значением ресурса

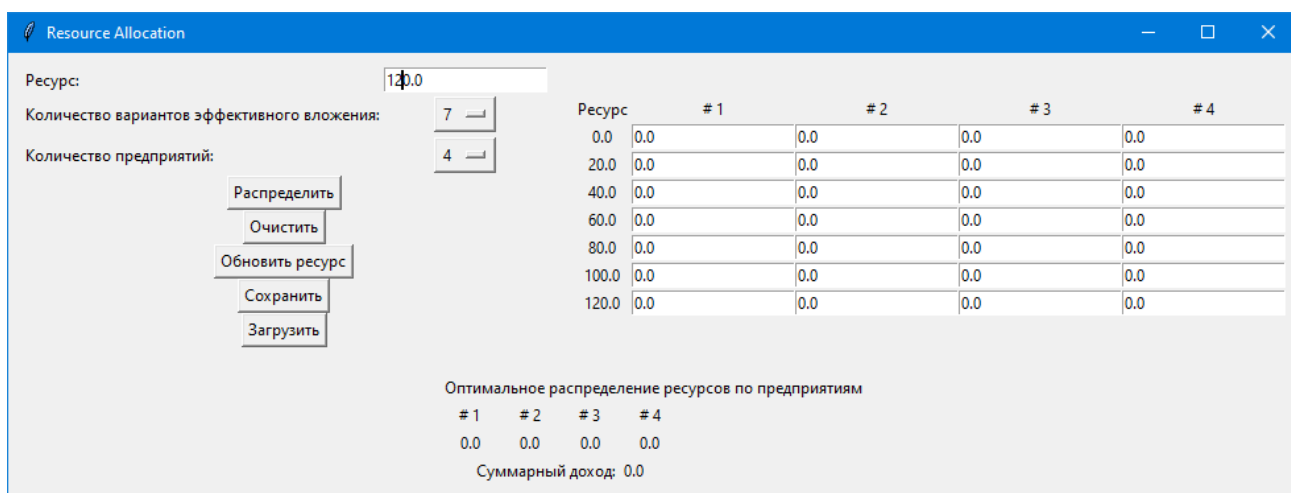


Рисунок 9 – Добавление нового варианта распределения

Resource Allocation

Ресурс: 120.0

Количество вариантов эффективного вложения: 7

Количество предприятий: 3

Распределить

Очистить

Обновить ресурс

Сохранить

Загрузить

Ресурс	# 1	# 2	# 3
0.0	0.0	0.0	0.0
20.0	0.0	0.0	0.0
40.0	0.0	0.0	0.0
60.0	0.0	0.0	0.0
80.0	0.0	0.0	0.0
100.0	0.0	0.0	0.0
120.0	0.0	0.0	0.0

Оптимальное распределение ресурсов по предприятиям

# 1	# 2	# 3
0.0	0.0	0.0

Суммарный доход: 0.0

Рисунок 10 – Удаление одного предприятия

Resource Allocation

Ресурс: 120.0

Количество вариантов эффективного вложения: 7

Количество предприятий: 3

Распределить

Ошибка

Заполните все ячейки корректными числами

OK

Ресурс	# 1	# 2	# 3
0.0	фывп	0.0	0.0
20.0	0.0	0.0	0.0
40.0	0.0	0.0	0.0
60.0	0.0	0.0	0.0
80.0	0.0	0.0	0.0
100.0	0.0	0.0	0.0
120.0	0.0	0.0	0.0

Оптимальное распределение ресурсов по предприятиям

# 1	# 2	# 3
0.0	0.0	0.0

Суммарный доход: 0.0

Рисунок 11 – Некорректное заполнение таблицы

Resource Allocation

Ресурс: 120.0

Количество вариантов эффективного вложения: 7

Количество предприятий: 3

Распределить

Очистить

Обновить ресурс

Сохранить

Загрузить

Ресурс	# 1	# 2	# 3
0.0	0.0	0.0	0.0
20.0	0.0	0.0	5
40.0	5	0.0	0.0
60.0	0.0	5	0.0
80.0	0.0	0.0	0.0
100.0	0.0	0.0	0.0
120.0	0.0	0.0	0.0

Оптимальное распределение ресурсов по предприятиям

# 1	# 2	# 3
40.0	60.0	20.0

Суммарный доход: 15.0

Рисунок 12 – Успешное чтение данных и выполнение алгоритма ДП

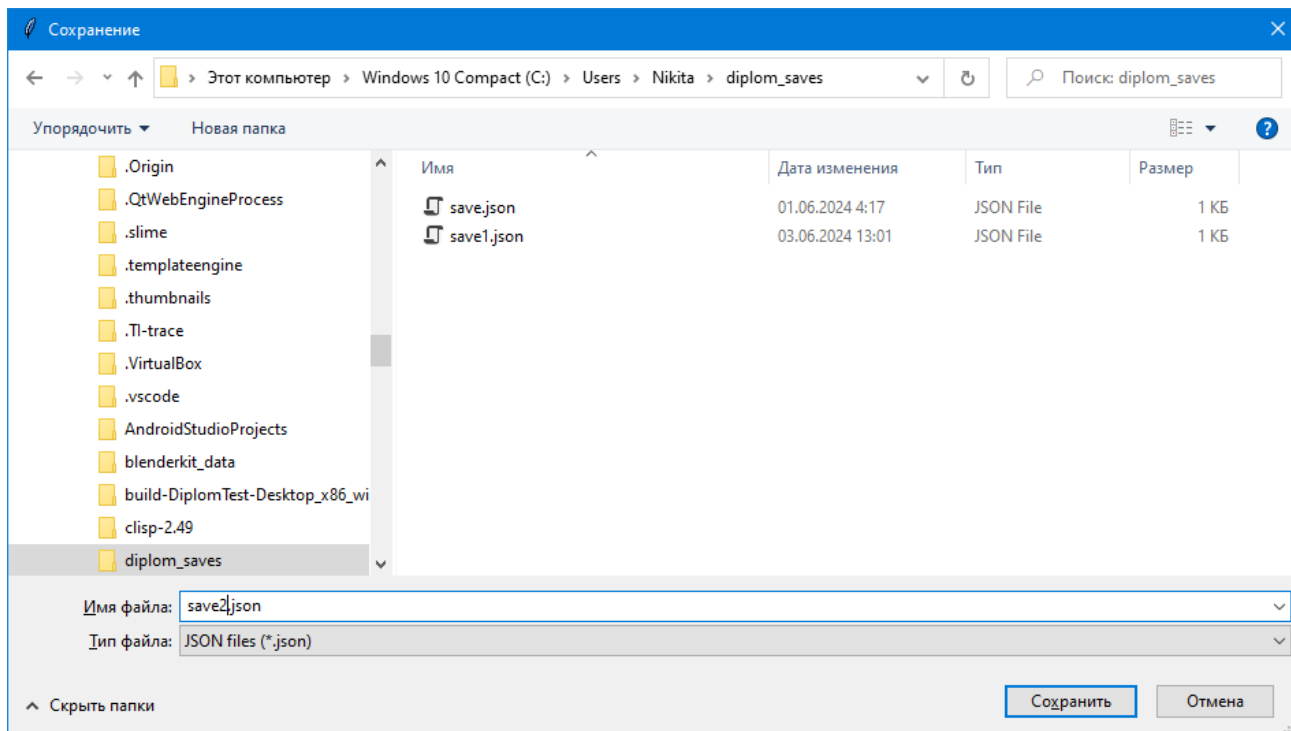


Рисунок 13 – Сохранение данных

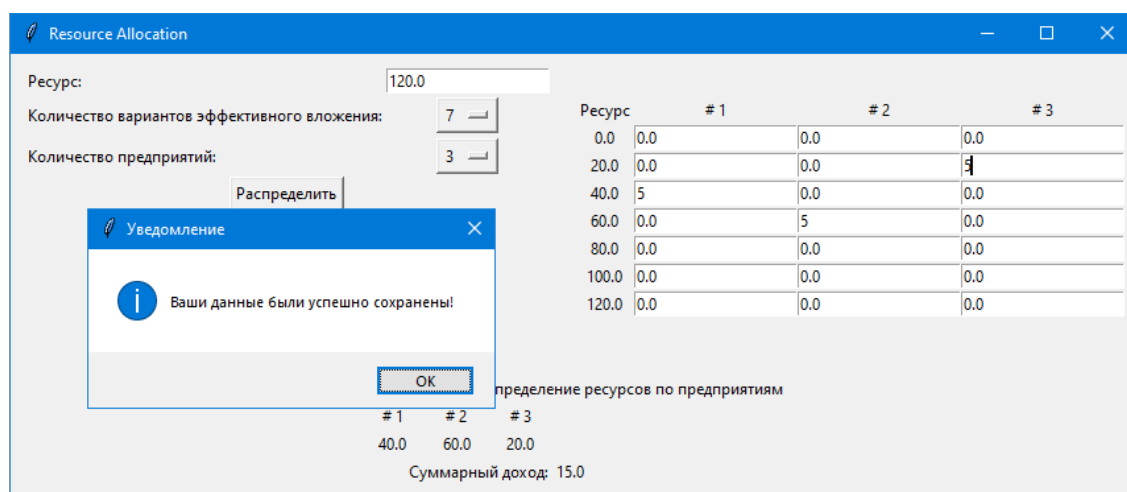


Рисунок 14 – Результат сохранения

Протестируем работу программы с исходными данными, приведенными в пункте 2.3 2 главы (см. рисунки 15-16).

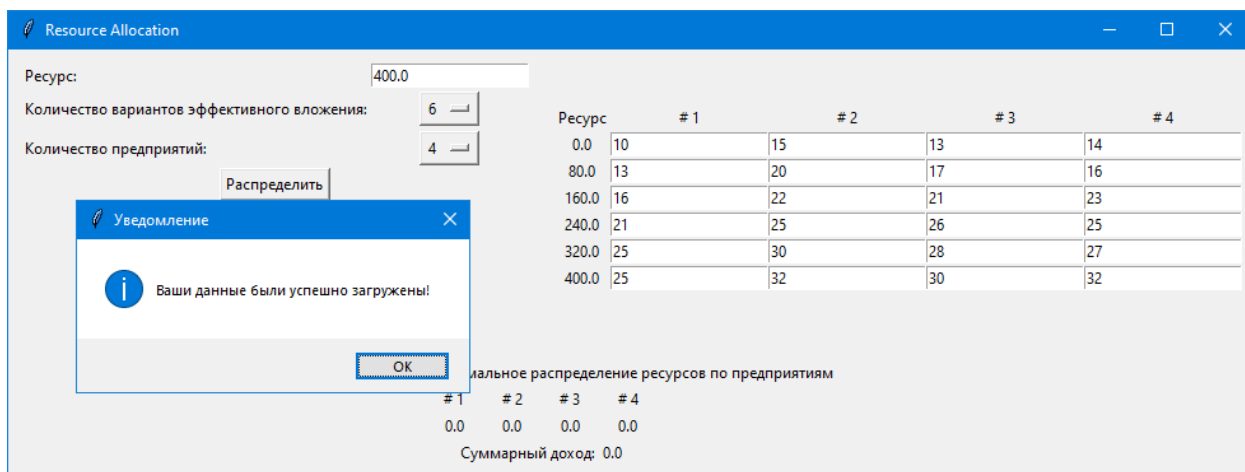


Рисунок 15 Загрузка входных данных

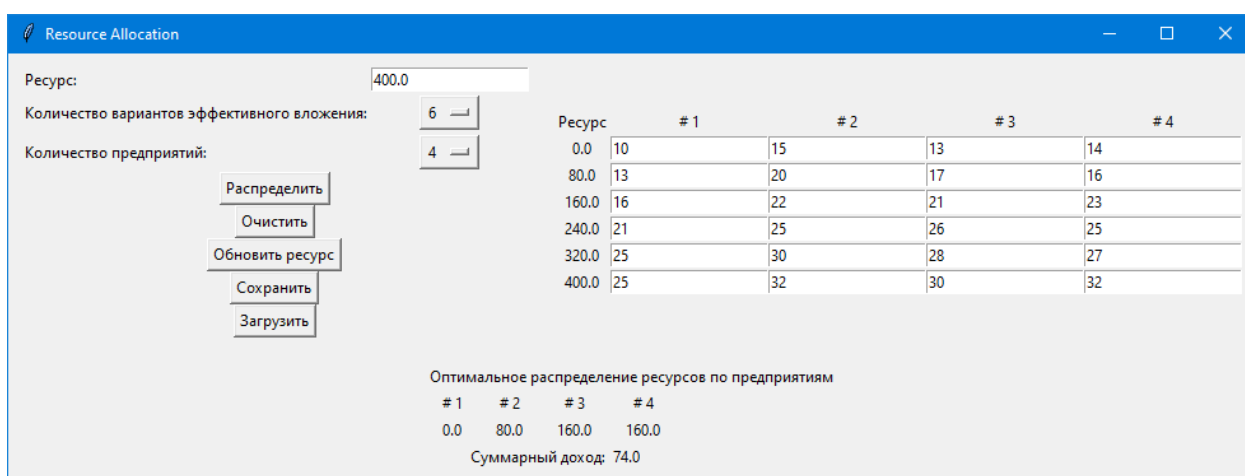


Рисунок 16 Результат

Как видно из Рисунка 16, результат работы алгоритма совпал с ручным расчетом.

На этом, демонстрация реализованного и разработка нового функционала заканчивается, т.к. минимальные требования, поставленные и продемонстрированные на УС диаграмме (см. Рисунок 3) были выполнены.

4 Исследование эффективности работы алгоритма динамического программирования

4.1 Краткое введение в существующие альтернативы

Для решения задачи распределения ограниченных средств S между N предприятиями производственного объединения, кроме метода динамического программирования, можно использовать следующие методы:

- Линейное программирование (симплекс-метод)
- Целочисленное программирование (метод ветвей и границ)
- Эвристические алгоритмы (эволюционные методы и жадный алгоритм)

Кратко рассмотрим каждый из вышеперечисленных методов, а также опишем на основе каких готовых решений или библиотек производилась их реализация.

Линейное программирование:

Линейное программирование – это метод математического моделирования, используемый для оптимизации (максимизации или минимизации) линейной целевой функции при наличии линейных ограничений. В задачах линейного программирования переменные принимают непрерывные значения, и функции прибыли являются линейными.

Целочисленное программирование

Целочисленное программирование – это вид математического программирования, где некоторые или все переменные задачи должны принимать целые значения. Этот тип программирования особенно полезен в случаях, когда решения должны быть дискретными, например, при распределении ресурсов, оптимизации маршрутов или решении задач планирования.

Поскольку наша оптимизационная задача задает табличные данные дискретно, то общие математические модели линейного и целочисленного

программирования практически не будут отличаться.

Сначала сформулируем общую задачу:

Максимизировать $S = \sum_{k=1}^n c_k t_k$ при ограничениях:

$$\begin{aligned} a_{11}t_1 + a_{21}t_2 + \dots + a_{k1}t_k + \dots + a_{n1}t_n &\geq b_1, \\ a_{12}t_1 + a_{22}t_2 + \dots + a_{k2}t_k + \dots + a_{n2}t_n &\geq b_2, \\ &\dots \\ a_{1l}t_1 + a_{2l}t_2 + \dots + a_{kl}t_k + \dots + a_{nl}t_n &\geq b_l, \quad (14) \\ &\dots \\ a_{1m}t_1 + a_{2m}t_2 + \dots + a_{km}t_k + \dots + a_{nm}t_n &\geq b_m, \\ t_1 \dots t_n &\geq 0 \end{aligned}$$

Где $c_1 \dots c_n$ – коэффициенты целевой функции, a_{kl} – коэффициенты ограничений, b_l – правые части ограничений, $t_1 \dots t_n$ – переменные, которые необходимо определить.

Очевидно, что в рамках сформулированной целевой задачи во 2 главе (все известные обозначения также берутся оттуда), коэффициентами целевой функции должна быть прибыль от вложения в предприятие, т.е. $c_{kl} = x_{kl}$. Пусть $p_{11} \dots p_{nm} = t_{11} \dots t_{nm}$ – набор бинарных переменных, которые необходимо определить (т. е. $p = 1$ или $p=0$). Тогда целевая функция S приобретет следующий вид:

$$S = \sum_{k=1}^n \sum_{l=1}^m x_{kl} p_{kl} \quad (15)$$

Для новой целевой функции ограничения примут следующий вид:

- Общие средства, вложенные во все предприятия, не должны превышать S .

$$\sum_{k=1}^n \sum_{l=1}^m l * \Delta * p_{kl} \leq S \quad (16)$$

- Для каждого предприятия выбирается ровно один уровень вложений (уровень вложений зависит от бинарной переменной).

$$\sum_{l=1}^m p_{kl} = 1, k = 1 \dots n$$

В рамках данной математической модели были реализованы алгоритмы линейного (симплекс-метод) и целочисленного (метод ветвей и границ) программирования при помощи библиотеки PuLP.

Библиотека PuLP в Python используется для моделирования и решения задач линейного программирования (ЛП) и целочисленного линейного программирования (ЦП). Она предоставляет интерфейс для различных решателей (solvers), которые непосредственно выполняют решение задач. Основные решатели, которые были использованы для сравнения:

CBC (Coin-or branch and cut)

- Это решатель по умолчанию в PuLP. CBC является свободным решателем для линейного и целочисленного программирования, разработанным проектом COIN-OR[18].
- Он хорошо интегрирован с PuLP и не требует установки дополнительных программ, если используется стандартная версия PuLP.
- Основан на алгоритме метода ветвей и границ

GLPK (GNU Linear Programming Kit)

- GLPK – это открытый решатель, который поддерживает линейное программирование и смешанное целочисленное программирование[19].
- Для использования GLPK с PuLP необходимо установить его отдельно и указать путь к решателю при решении задачи.
- Способен решать задачи линейного программирования симплекс-методом.

Эвристические методы

Эвристические методы – это подходы к решению сложных оптимизационных задач, которые не гарантируют нахождения глобально оптимального решения, но могут найти достаточно хорошее решение за

разумное время. Эти методы особенно полезны для задач большого размера или сложных систем, где традиционные методы оптимизации могут быть неэффективными.

В рамках сравнения были рассмотрены следующие эвристические методы.

- Эволюционный алгоритм.
- Жадный алгоритм.

Эволюционные алгоритмы

Эволюционные алгоритмы – это класс стохастических методов оптимизации, вдохновленных механизмами естественного отбора и генетики. Основная идея этих алгоритмов заключается в создании и эволюции популяции возможных решений с целью нахождения наиболее пригодного решения.

Основные компоненты эволюционных алгоритмов.

1. Представление решения.

Решения кодируются в виде хромосом (обычно в виде строк чисел или символов).

2. Инициализация популяции.

Начальная популяция решений создается случайным образом или с использованием некоторой эвристики.

3. Оценка пригодности (fitness function).

Каждое решение оценивается с точки зрения его пригодности для данной задачи (например, с помощью целевой функции).

4. Селекция (выбор родителей).

Отбор решений для создания нового поколения на основе их пригодности. Популярные методы включают рулетку, турнирный отбор и ранговый отбор.

5. Кроссовер (рекомбинация).

Комбинация двух или более родительских решений для создания

одного или нескольких потомков.

6. Мутация:

Внесение случайных изменений в решения для поддержания генетического разнообразия в популяции.

7. Замена.

Обновление популяции путем замены старых решений новыми, более пригодными.

8. Завершение.

Алгоритм завершает работу, когда достигается предопределенное условие остановки (например, достижение заданного числа поколений или стабилизация значения целевой функции).

Для реализации генетического алгоритма использовалась библиотека DEAP

Библиотека **DEAP** (Distributed Evolutionary Algorithms in Python) представляет собой мощный инструмент для разработки и реализации эволюционных алгоритмов, включая генетические алгоритмы[20].

Основные компоненты DEAP:

1. **Creator:** Используется для создания типов, таких как особи и популяции. Это позволяет определять структуры данных для особей, которые будут использоваться в процессе эволюции.

2. **Base:** Включает основные функции и классы, такие как **Toolbox**, которые предоставляют механизм для регистрации функций, используемых в эволюционном процессе (например, функции отбора, кроссовера, мутации).

3. **Tools:** Содержит множество готовых к использованию инструментов для реализации различных этапов генетических алгоритмов, таких как селекция, кроссовер и мутация.

4. **Algorithms:** Включает готовые алгоритмы для выполнения эволюционного процесса. Например, функция **eaSimple** реализует простой эволюционный алгоритм.

5. Benchmarks: Содержит набор стандартных тестовых задач для оценки производительности эволюционных алгоритмов.

Генетический алгоритм разрабатывался следующим образом:

Инициализация.

- Создание типов данных для индивидуумов и их фитнес-функций.
- Формирование индивидов производилось следующим образом, случайным образом формировалась политика распределения средств между k предприятиями, а оставшиеся варианты до n приравнивались к 0.
- Определение параметров генетического алгоритма, таких как размер популяции, вероятность кроссовера и мутации, и количество поколений.

Определение оценочной функции.

- Функция `evalProfit` вычисляет общую прибыль для данного распределения средств (индивидуума).
- Если общее вложение превышает доступные средства S , функция возвращает штраф (отрицательное значение), чтобы исключить такие решения из рассмотрения.

Операторы генетического алгоритма:

- `mate`: двухточечный кроссовер.
- `mutate`: мутация с равномерным распределением.
- `select`: турнирный отбор.

Запуск алгоритма:

- Инициализация начальной популяции и выполнение генетического алгоритма с заданными параметрами.
- Использование `eaSimple`, стандартного алгоритма из библиотеки DEAP, для выполнения эволюционного процесса.

Вывод результатов:

- Определение и вывод наилучшего найденного решения и

соответствующей прибыли.

Жадные алгоритмы

Жадные алгоритмы – это класс алгоритмов, которые строят решение задачи шаг за шагом, на каждом этапе принимая локально оптимальное решение с надеждой на то, что конечное решение будет глобально оптимальным. Философия жадных алгоритмов заключается в том, чтобы на каждом этапе выбирать тот вариант, который кажется наилучшим в данный момент, не рассматривая более глобальные последствия этих решений.

Жадный алгоритм для нашей задачи был реализован самостоятельно без использования библиотек и заключался в следующем – выбирать на каждом шаге то предприятие и уровень инвестиций, которые дают наибольший прирост прибыли на единицу вложенных средств. Алгоритм продолжает распределять средства до тех пор, пока все средства не будут распределены.

4.2 Методология исследования

В данной работе было принято решение следовать методологии эмпирического исследования для оценки и сравнения эффективности различных алгоритмов решения задачи оптимального распределения инвестиций. Эмпирическое тестирование включает в себя проведение экспериментов и сбор данных для анализа и сравнения характеристик производительности алгоритмов в различных сценариях. Ниже описаны основные шаги, которые были выполнены в рамках этой методологии.

Определение целей и метрик эффективности.

Основной целью тестирования являлась оценка времени выполнения, максимальной прибыли и использования памяти для каждого из алгоритмов. Таким образом, метрики тестирования включают в себя:

- Время выполнения (в секундах) для каждого тестового случая.

- Максимальная прибыль, полученная в результате распределения инвестиций для каждого тестового случая.

Выбор и реализация алгоритмов.

Были реализованы несколько алгоритмов для решения задачи оптимального распределения инвестиций, включая:

- Жадный алгоритм (Greedy Algorithm)
- Генетический алгоритм (Genetic Algorithm)
- Метод ветвей и границ (Branch and Bound)
- Симплекс-метод (Simplex Method)
- Динамическое программирование (Dynamic Programming)

Создание тестовых случаев.

Была разработана функция *generate_random_data* для генерации случайных тестовых данных, представляющих прибыль при различных уровнях инвестиций для каждого предприятия (по сути, случайная генерация Таблицы №1). Определен набор групп тестов с различными параметрами, такими как общий бюджет (S), количество предприятий (N) и шаг инвестиций (Δ) для каждого тестового случая.

Легкая тестовая группа:

- Параметры:
 - Количество тестов: 6
 - $S_0 = 200, N_0 = 10, \Delta_0 = 100$
 - $S_i = S_{i-1} + 175, N_i = N_{i-1} + 2, S_i = S_{i-1} + 175 \quad \Delta_i = \Delta_{i-1} - 25, i = 1, 3, 5$

Реалистичная тестовая группа:

- Параметры:
 - Количество тестов: 8
 - $S_0 = 240, N_0 = 10, \Delta_0 = 120$
 - $S_i = S_{i-1} * 2, N_i = N_{i-1} + 2, S_i = S_{i-1} + 175 \quad \Delta_i = \Delta_{i-1} - 20, i = 2, 4, 6$

Тяжелая тестовая группа:

- Параметры:
 - Количество тестов: 10
 - $S_0 = 100, N_0 = 5, \Delta_0 = 10$
 - $S_i = S_{i-1} * \text{rand}(1,9) * (i + 1), N_i = N_{i-1} + (i + 1) + \text{rand}(2,10), \Delta_i = \Delta_{i-1} * (i + 1), i = 1, 3, 5, 7, 9$

На основе этих параметров и осуществляется работа функции *generate_random_data*

Запуск и сбор данных.

Написана функция *compare_algorithms*, которая в зависимости от выбранного теста запускала каждый алгоритм несколько раз, собирая данные о времени выполнения, максимальной прибыли и использовании памяти.

Анализ данных и визуализация результатов.

Написана функции *plot_time_results*, *plot_profit_result*, *plot_time_vs_variables*, *plot_variables* для построения графиков, сравнивающих время выполнения, максимальную прибыль, количество переменных и зависимость времени от количества переменных в таблице.

Также реализована функция *print_results* для вывода тех же результатов, но в виде таблицы.

Примечание.

Реализацию всех вышеперечисленных функций можно найти в приложении

Практически все параметры генетического алгоритма будут оставаться постоянными – вероятность мутации равна 35%, вероятность кроссовера равна 65%, количество индивидов в популяции = 300. Будем менять только число поколений для отслеживания влияния этого параметра на точность решения и время выполнения

Остальные алгоритмы не требуют настройки параметров.

4.3 Проведение исследования

В рамках нашего исследования для каждой группы тестов (легкий, реалистичный и тяжелый), запускаемых в соответствии с приведенными в пункте 4.2 параметрами, выводится от 3 до 4 графиков, смысл которых раскрыт ниже.

- Сравнение времени для каждого алгоритма, где на оси Ох отображены номера выполняемых тестов в рамках своей группы, а на оси Оу – время выполнения (в секундах) алгоритма в секундах для каждого теста.
- Сравнение максимальной прибыли для каждого алгоритма, где на оси Ох отображены номера выполняемых тестов в рамках своей группы, а на оси Оу – максимальная полученная прибыль в рамках сгенерированной задачи(таблицы) для каждого теста.
- Количество переменных, где на оси Ох отображены номера выполняемых тестов в рамках своей группы, а на оси Оу – количество переменных в таблице, используемой для этого теста.
- Зависимость времени от количества переменных, где на оси Ох отображено количество переменных, а на оси Оу – время выполнения (в секундах) алгоритма при таком количестве переменных

Также выводятся вспомогательные таблицы, которые по возможности будут продублированы в наших исследованиях.

Для начала запустим программу легкой группы тестов. Число поколений ГА = 50 и не будем выводить график:

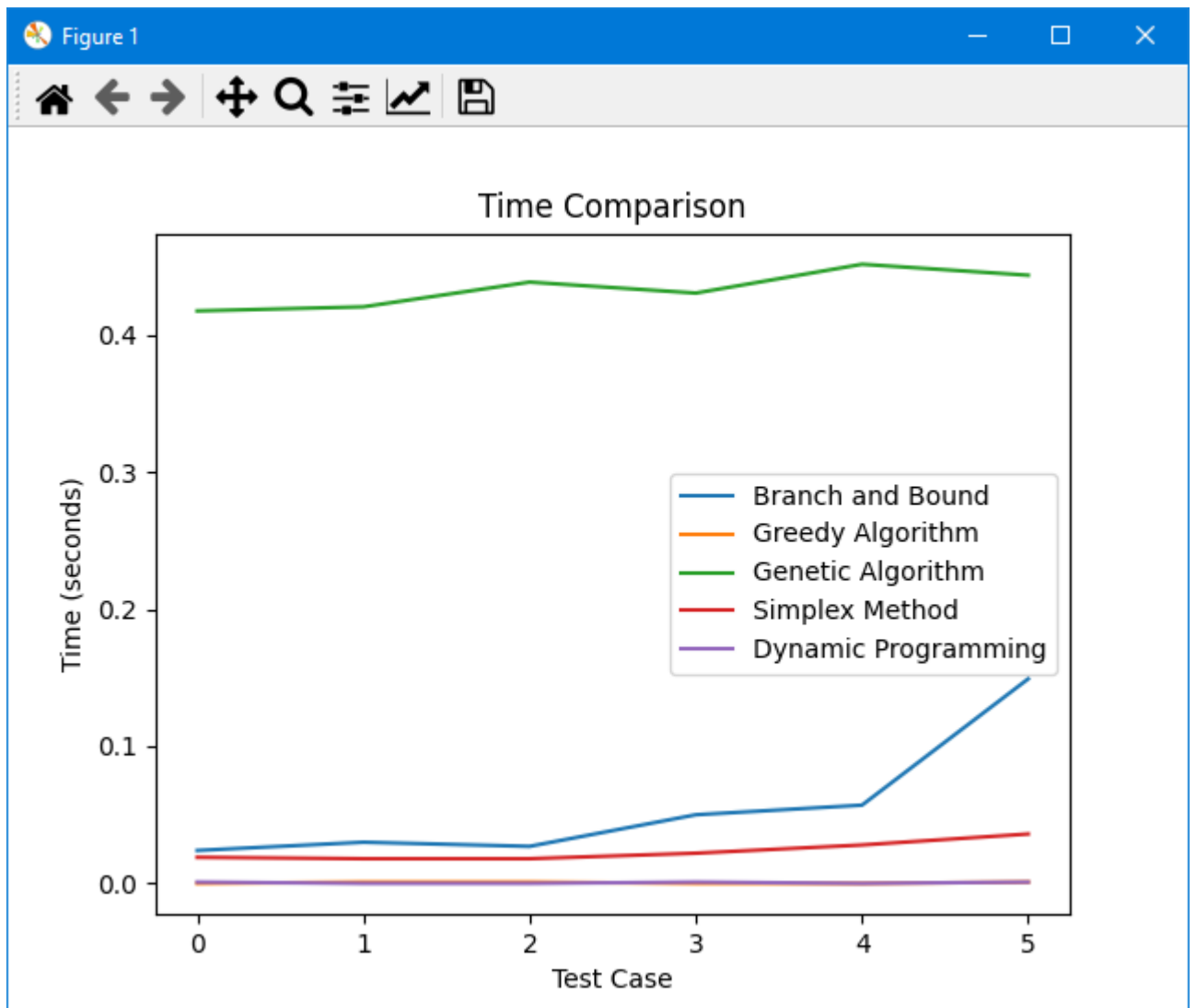


Рисунок 17 – Зависимость времени от номера теста

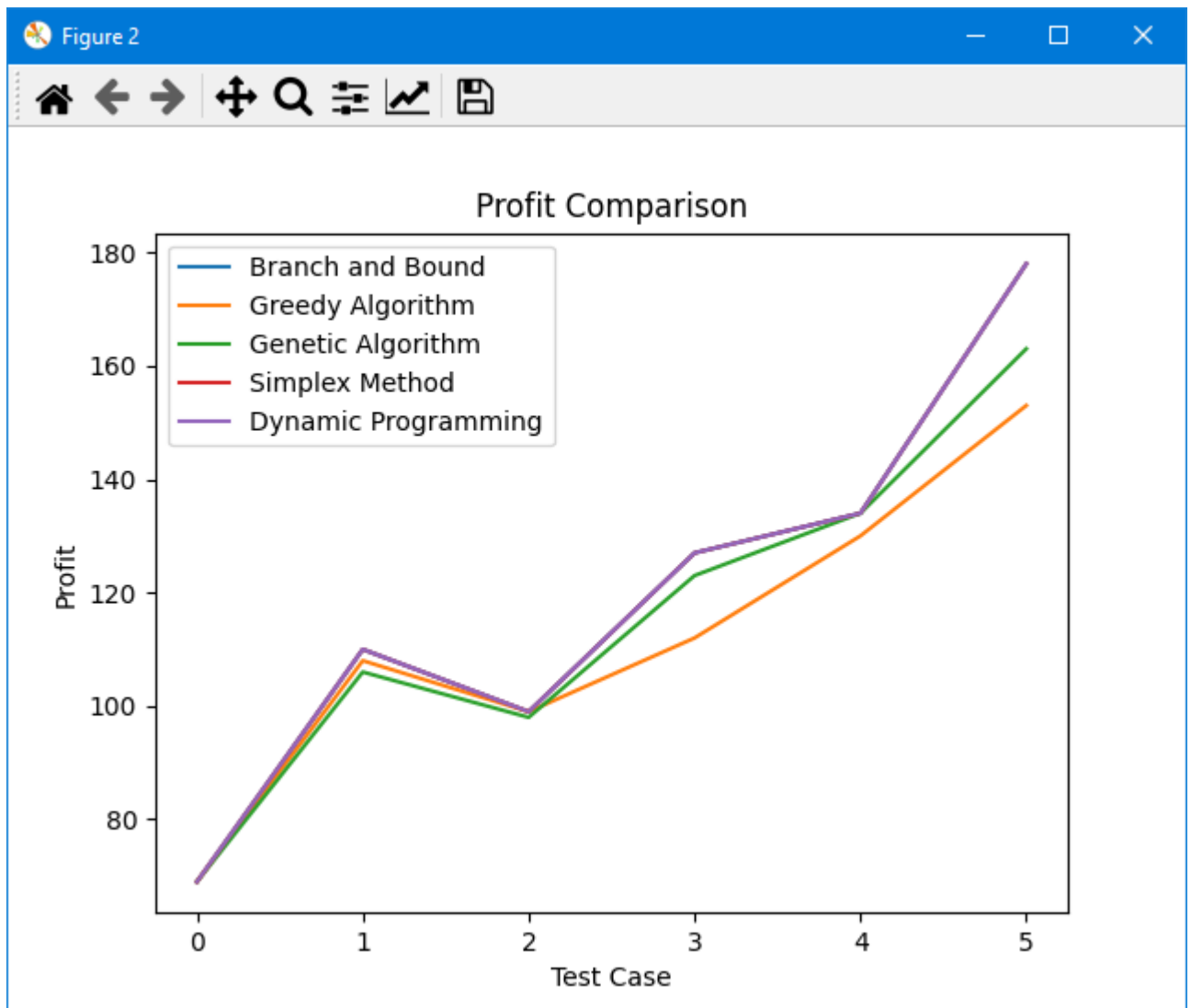


Рисунок 18 – Зависимость прибыли от номера теста

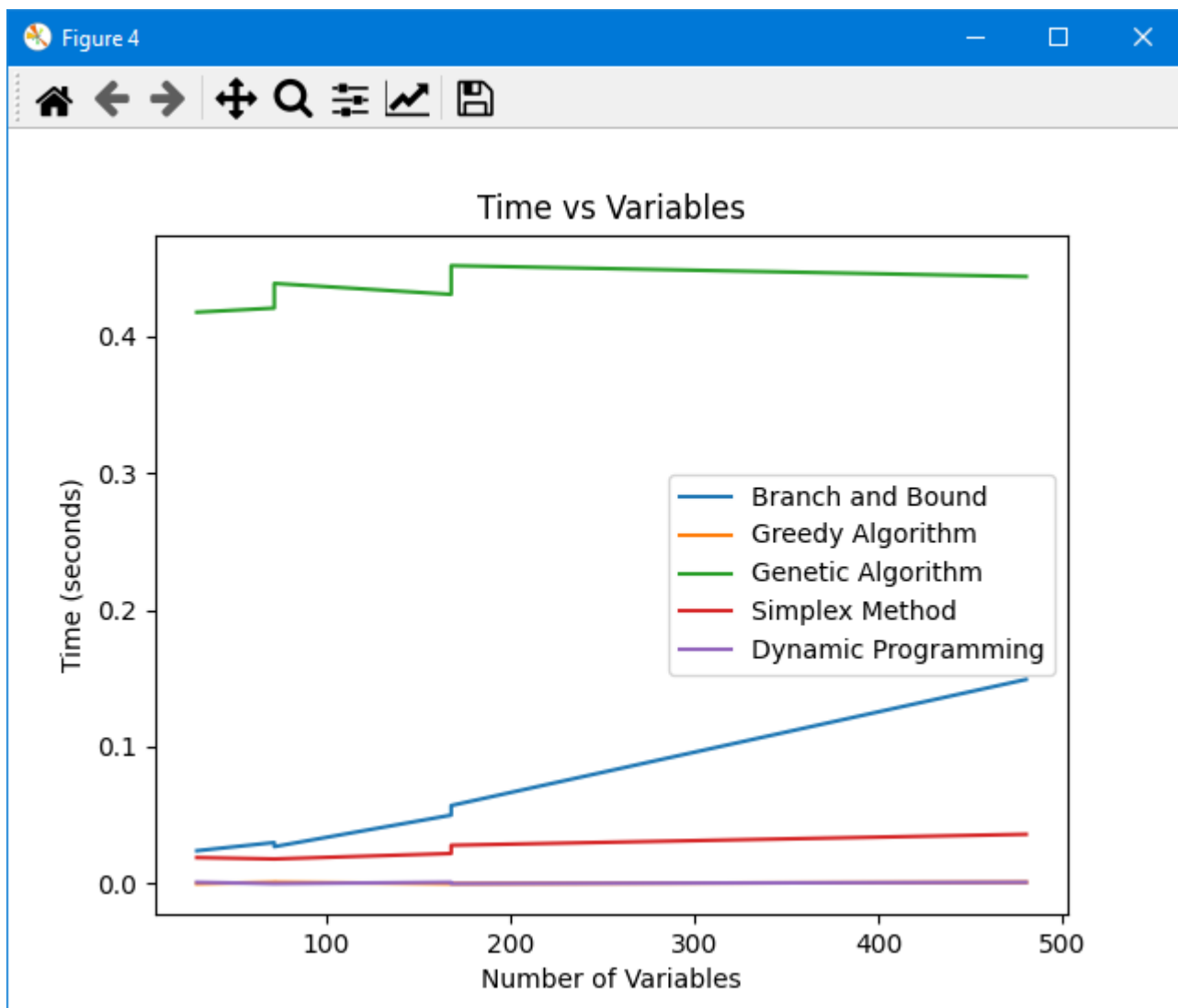


Рисунок 19 – Зависимость времени от количества переменных

Результаты отображенные на рисунках 16-19 подтверждают теоретические основы – при небольшом количестве параметров точность решения достаточно высока даже у эвристических методов. А алгоритм динамического программирования ничем не уступает точным математическим решателям для ЛП и ЦП в плане точности, при этом выигрывает во времени, однако все алгоритмы решают задачу менее чем за секунду, поэтому для получения более значимых результатов нужно переходить к следующей группе тестов.

Для реалистичной группы тестов пока оставим размер популяции = 50. Результаты работы «реалистичных» тестов:

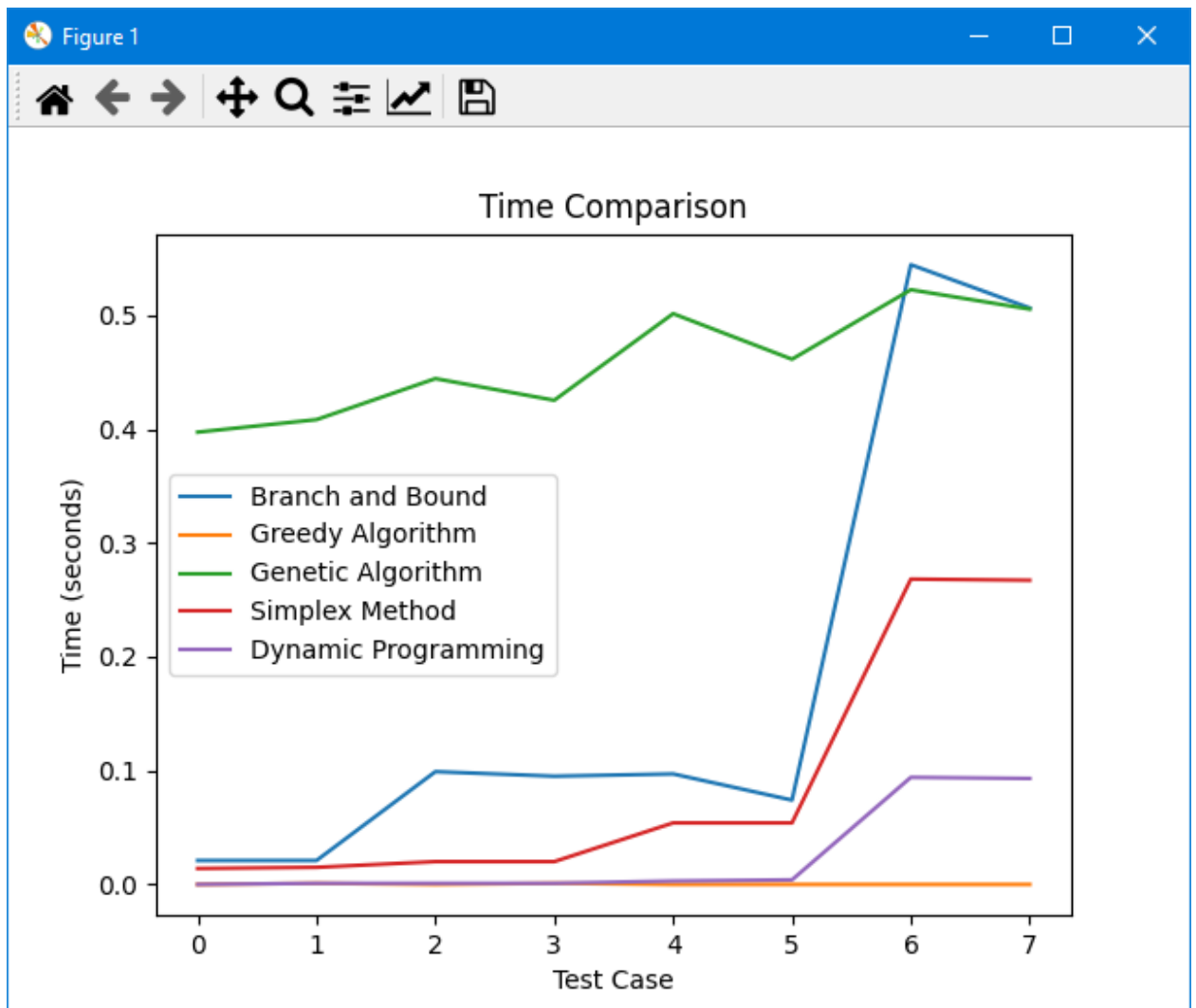


Рисунок 20 – Зависимость времени от номера теста

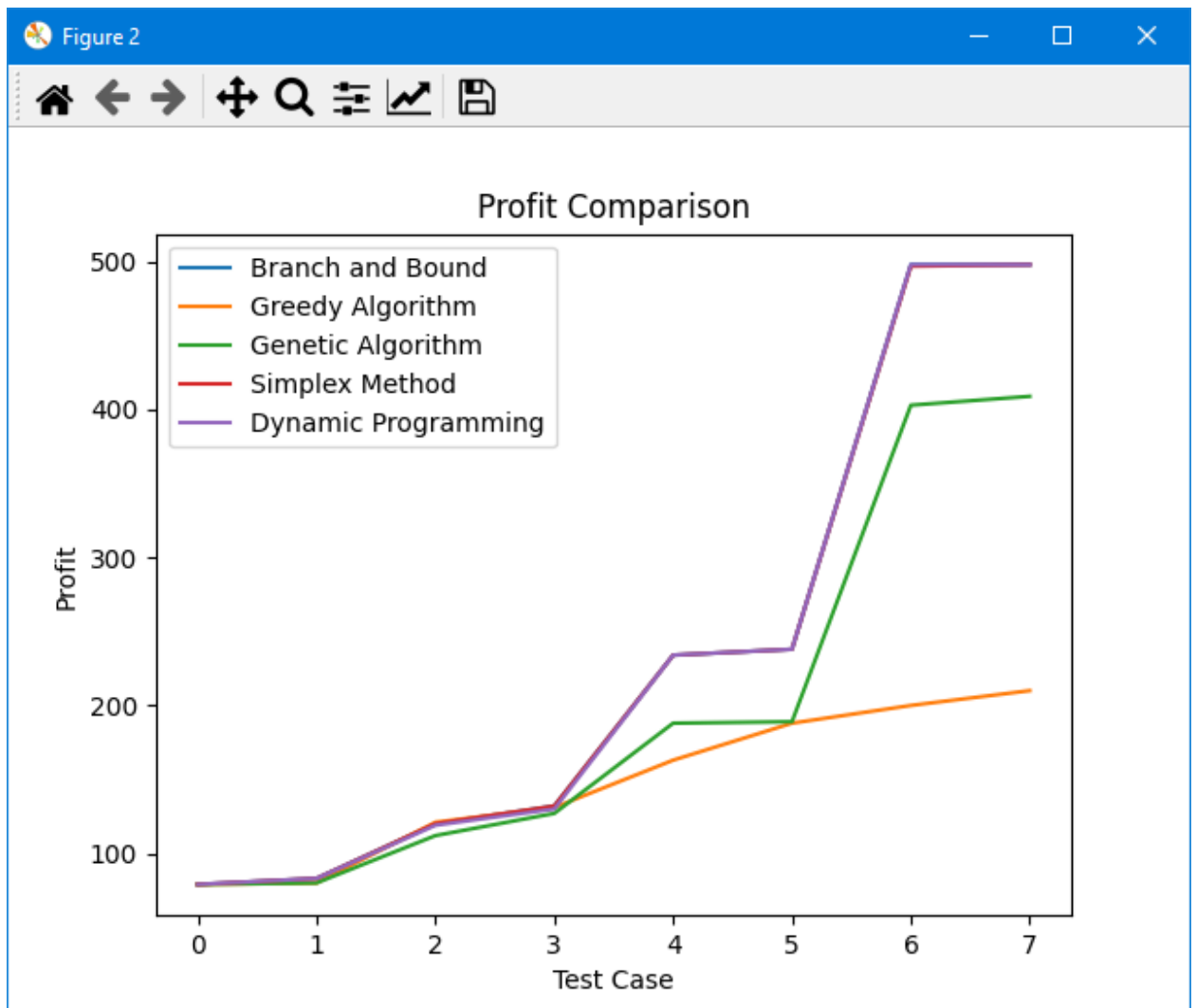


Рисунок 21 – Зависимость прибыли от номера теста

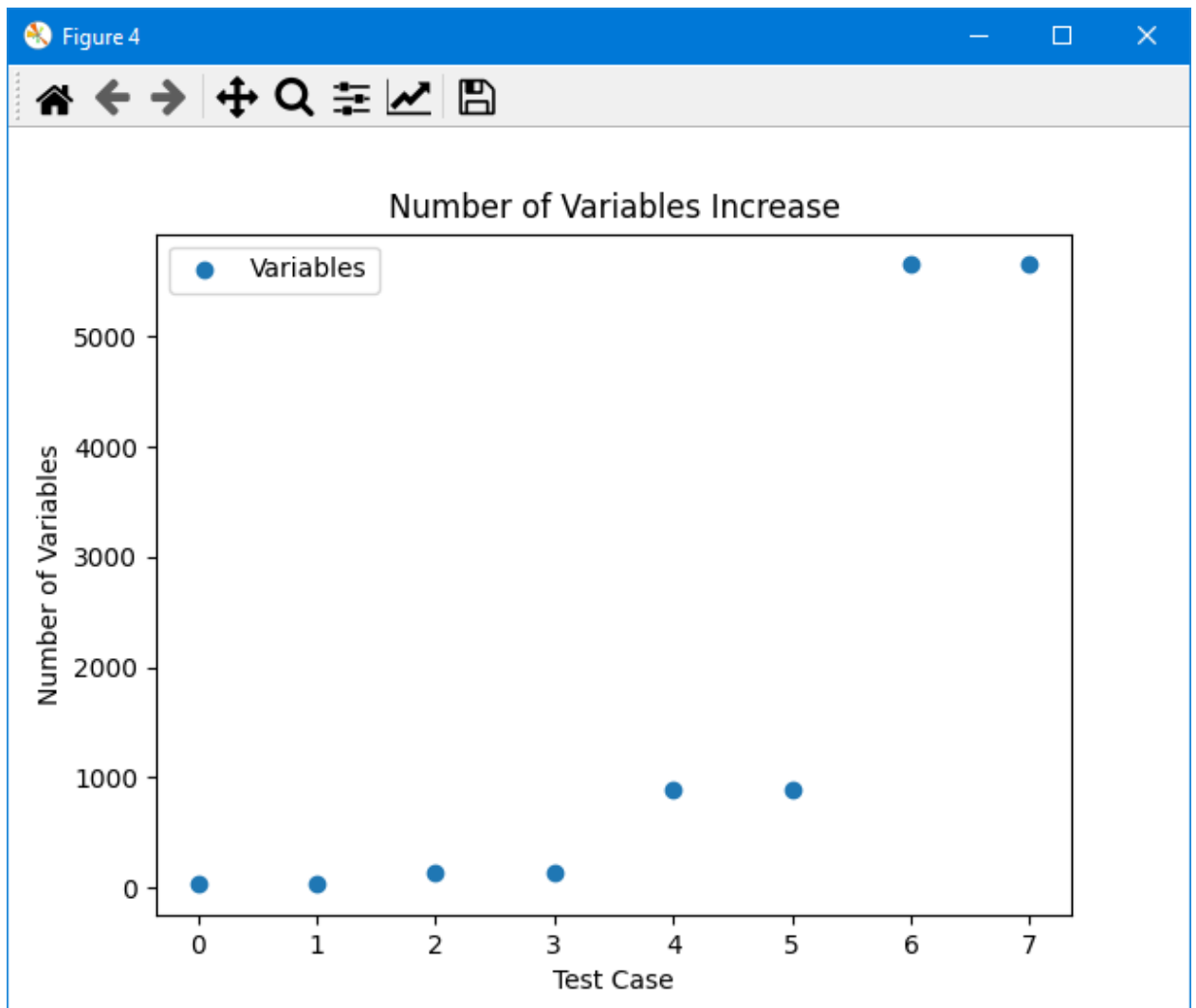


Рисунок 22 График зависимости количества переменных от номера теста

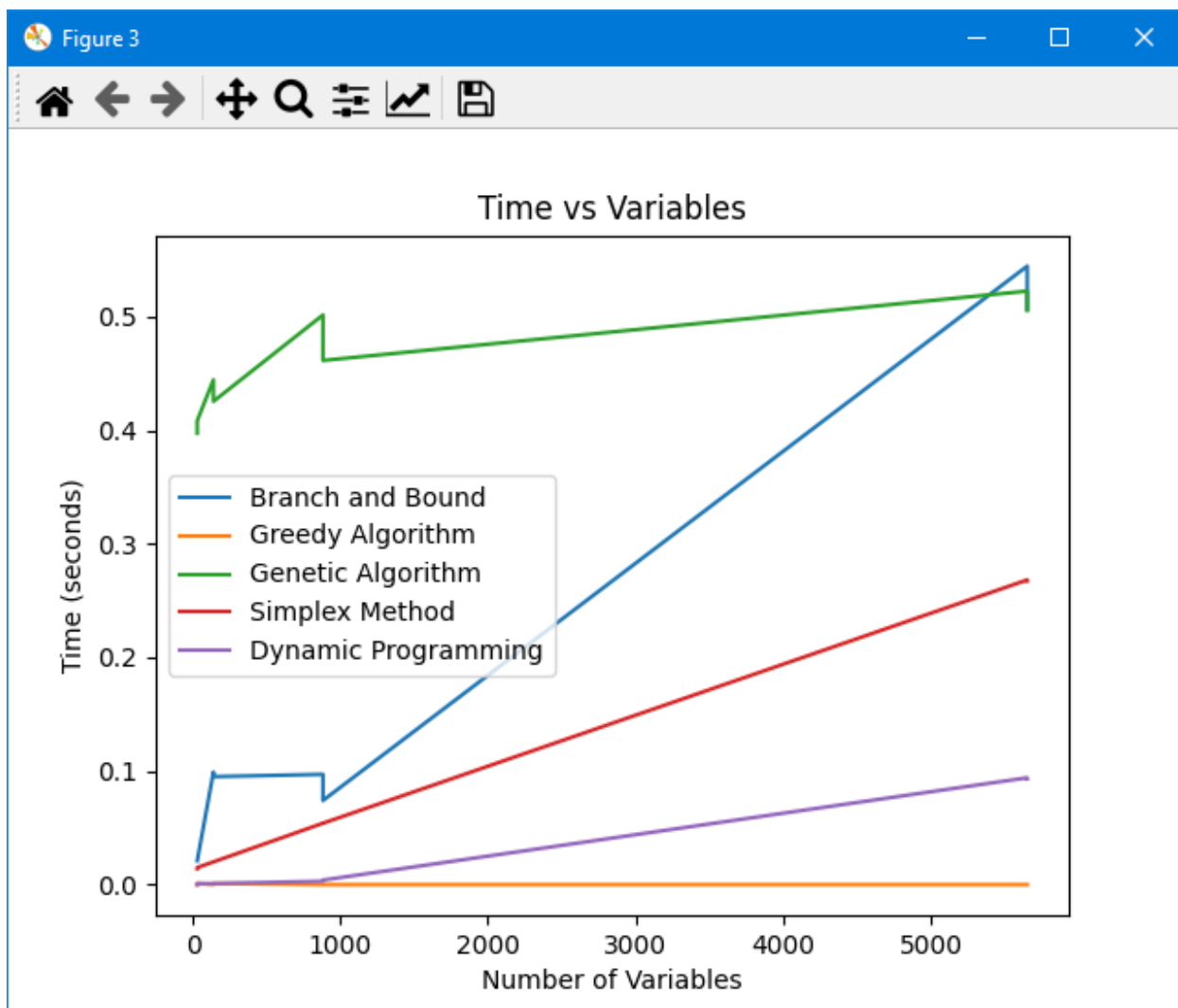


Рисунок 23 – Зависимость времени от количества переменных

Как видно из новых тестов и рисунков 20-23, с увеличением роста переменных начинает расти и время выполнения. Генетический алгоритм уже на этом этапе оказался быстрее метода ветвей и границ на десятки миллисекунды при количестве переменных равном 5624. При этом его субоптимальное решение можно улучшить, увеличив количество поколений.

Что же касается алгоритма динамического программирования, он все еще остается самым выигрышным в сравнении с аналогами, поскольку динамический шаг и таблица все еще имеют не слишком большой размер, что позволяет сохранять точность решения и скорость выполнения. При анализе вспомогательных таблиц, было выявлено, что среднее время выполнения на этом этапе для ДП составило около 25 миллисекунд, что

быстрее ближайшего по качеству симплекс метода практически в 4 раза. (см. рисунки 24-25).

Results for Dynamic Programming:

Test Case	Execution Time (s)	Max Profit	Number of Variables
1	0	79	30
2	0.00100064	83	30
3	0.00100064	119	140
4	0.00100136	130	140
5	0.00300384	234	882
6	0.00400472	238	882
7	0.0940855	498	5654
8	0.0930839	498	5654

Average time (s): 0.024647563695907593

Рисунок 24 Результаты реалистичных тестов для ДП

Results for Simplex Method:

Test Case	Execution Time (s)	Max Profit	Number of Variables
1	0.0140126	79	30
2	0.0150142	83	30
3	0.0200183	120	140
4	0.0200183	132	140
5	0.0540485	234	882
6	0.054049	238	882
7	0.268244	497	5654
8	0.267243	498	5654

Average time (s): 0.08908101916313171

Рисунок 25 Результаты реалистичных тестов для симплекс-метода

Жадный алгоритм же демонстрирует серьезную просадку точности решения, однако остается самым быстрым, в среднем выполняя расчет за 2 микросекунды, что в 2000 раз быстрее 0,5 секунды для генетического алгоритма.

Для трудного теста все еще оставим размер популяции =50. Результаты работы «трудных» тестов и используем весь доступный нам инструментарий:

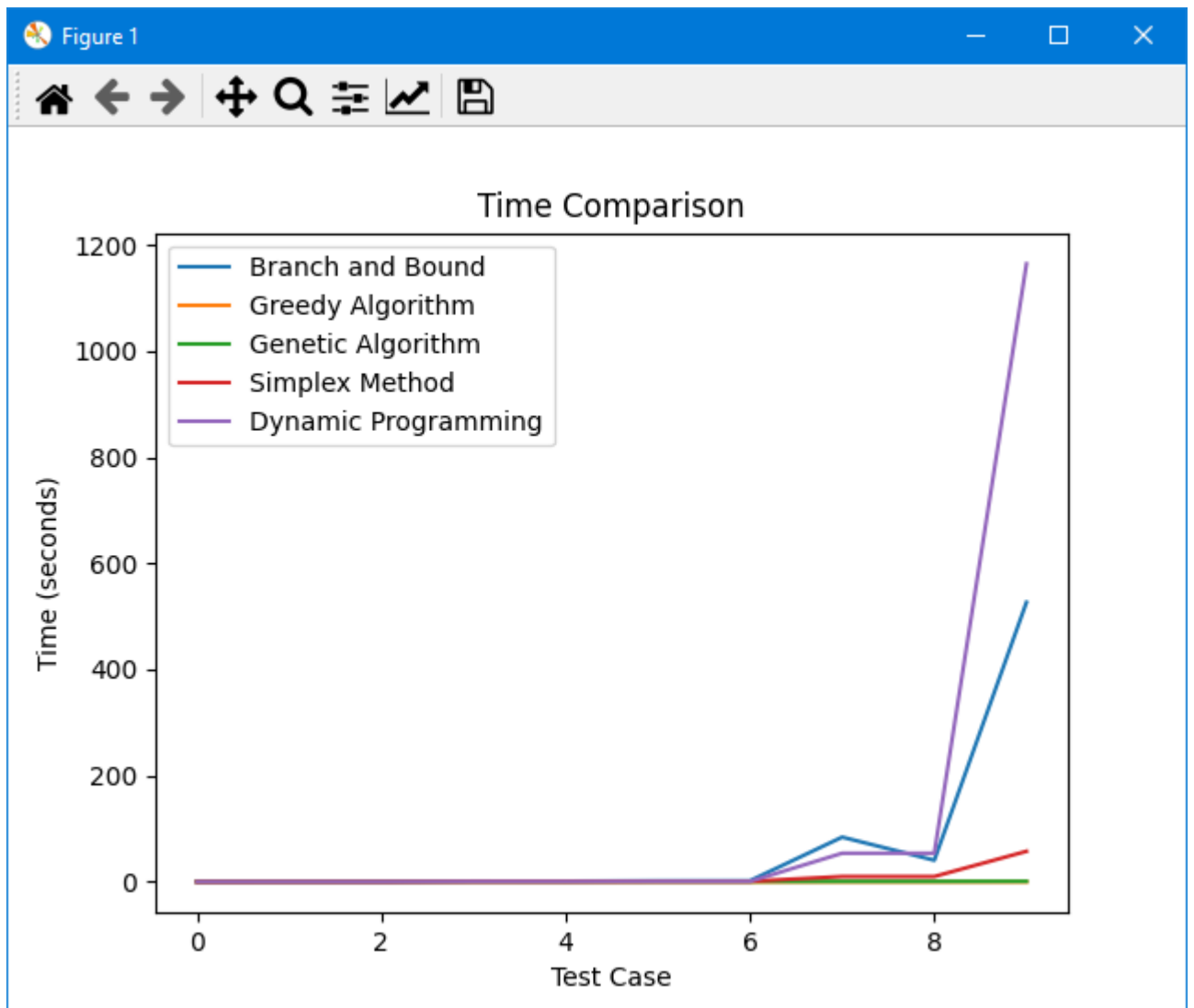


Рисунок 26 – Зависимость времени от номера теста

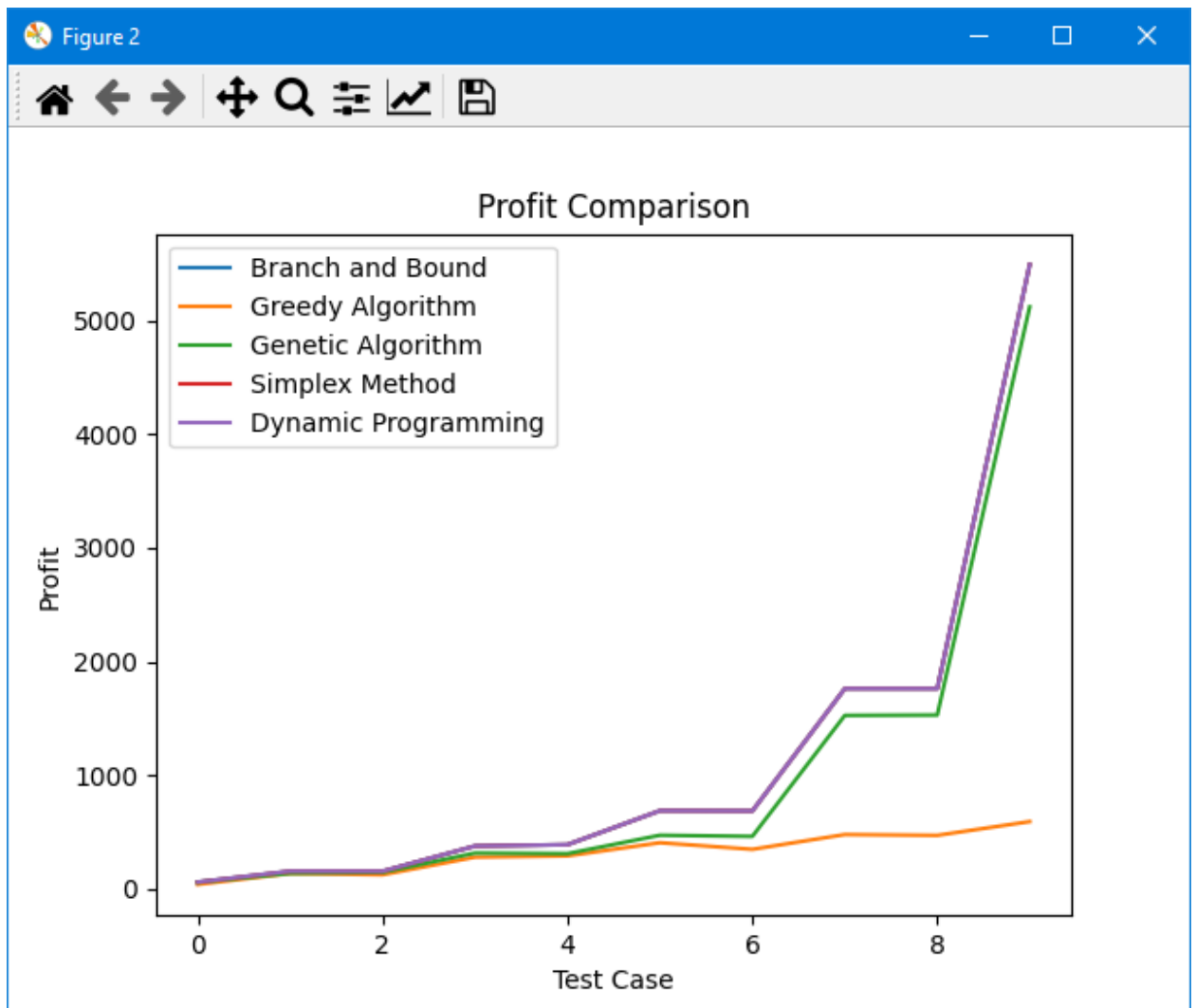


Рисунок 27 – Зависимость прибыли от номера теста

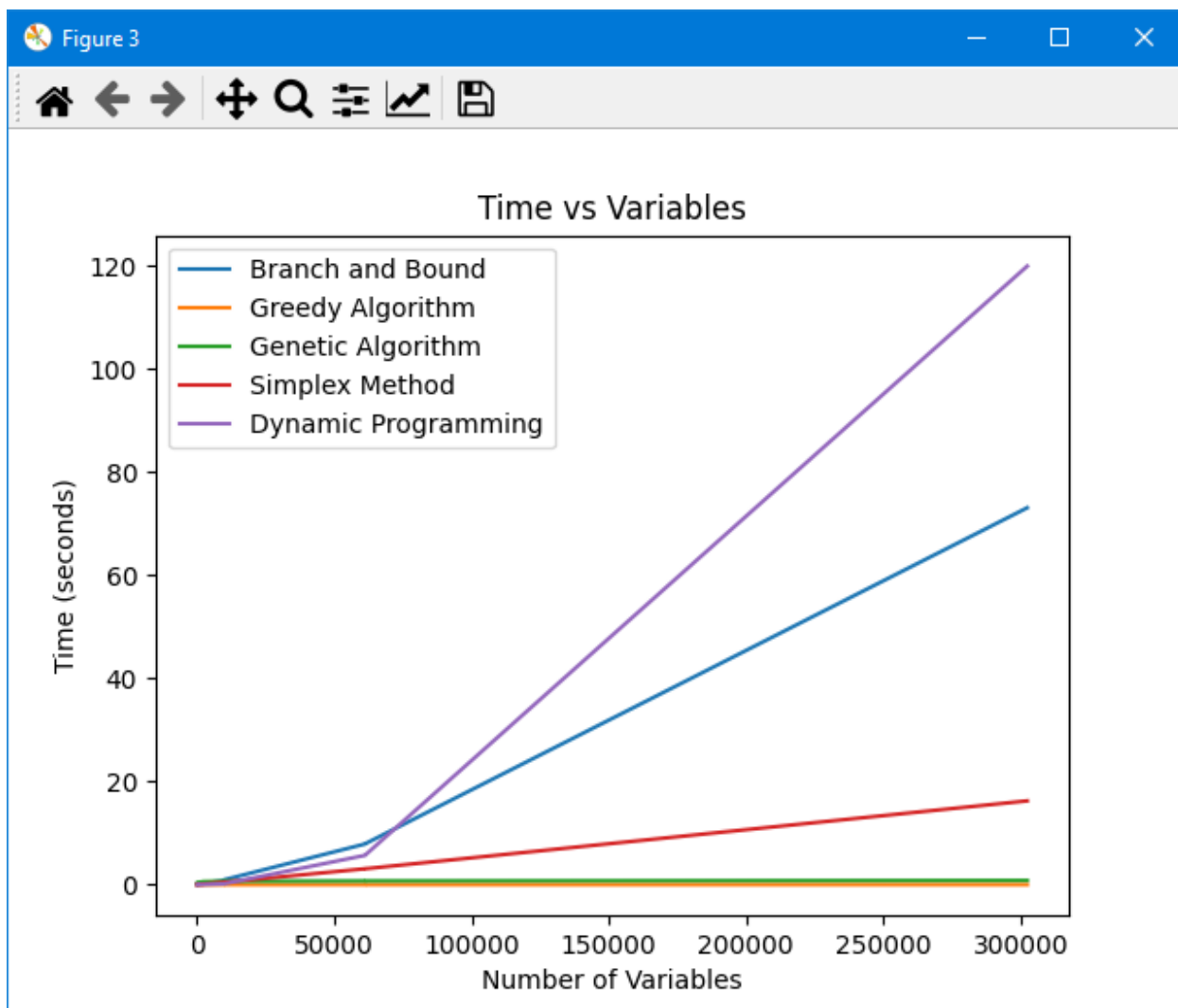


Рисунок 28 – Зависимость времени от количества переменных

Как видно из рисунков 26-28, при большом количестве элементов в таблице, полиномиальная сложность алгоритма динамического программирования начинает проявлять себя, и, без дополнительных модификаций алгоритма, время выполнения становится непомерно большим по сравнению с аналогами. Так, при росте количества переменных всего в ≈ 5 раз на 8-9 шагах (с 61251 до 302463), время выполнения, в свою очередь, выросло в 24 раза (с 5 до 120 секунд). На 6-7 шагах при росте переменных в ≈ 6 раз (9881 с до 61251) время выполнения выросло в 40 раз (с 0,15 до 6 секунд). Среднее время выполнения составило 13 секунд.

Генетический же алгоритм практически приближается к оптимальному

решению и выполняет расчёт быстрее других представленных алгоритмов. Время его выполнения зависит лишь от количества поколений, потому что за все шаги теста от 55 переменных к 302463 время его выполнения возросло всего лишь в 2 раза и все еще составляло меньше секунды (в среднем около 0,6 с).

Симплекс-метод в среднем тратит около 2 секунд на поиск решения, выражая линейный характер возрастания зависимости для времени выполнения от количества переменных. Так при росте на 8-9 шагах с 61251 до 302463 переменных, время выполнения возросло с 3.0798 с до 16.2408 с, что в обоих случаях равно увеличению в ≈ 5 . На 6-7 шагах – с 9881 до 61251 и с 0.458416 с до 3.0828 с соответственно, что можно расценить как возрастание в ≈ 6 .

Метод ветвей и границ в целом по динамике схож с методом ДП, но за счет более эффективной программной реализации решателя оказался быстрее в тяжелых случаях. Так для перехода 8-9, время выполнения возросло в ≈ 10 раз, а для шагов 6-7 – ≈ 8 раз со средним временем выполнения в 9 секунд. Разница в выигрыше по времени по сравнению с ДП будет только возрастать при дальнейшем увеличении размерности задачи.

Однако, при приближении графиков видно, что в пределах десятков тысяч элементов, алгоритм динамического программирования справляется быстрее, чем ЦП методом ветвей и границ (см. рисунок 29 и 30):

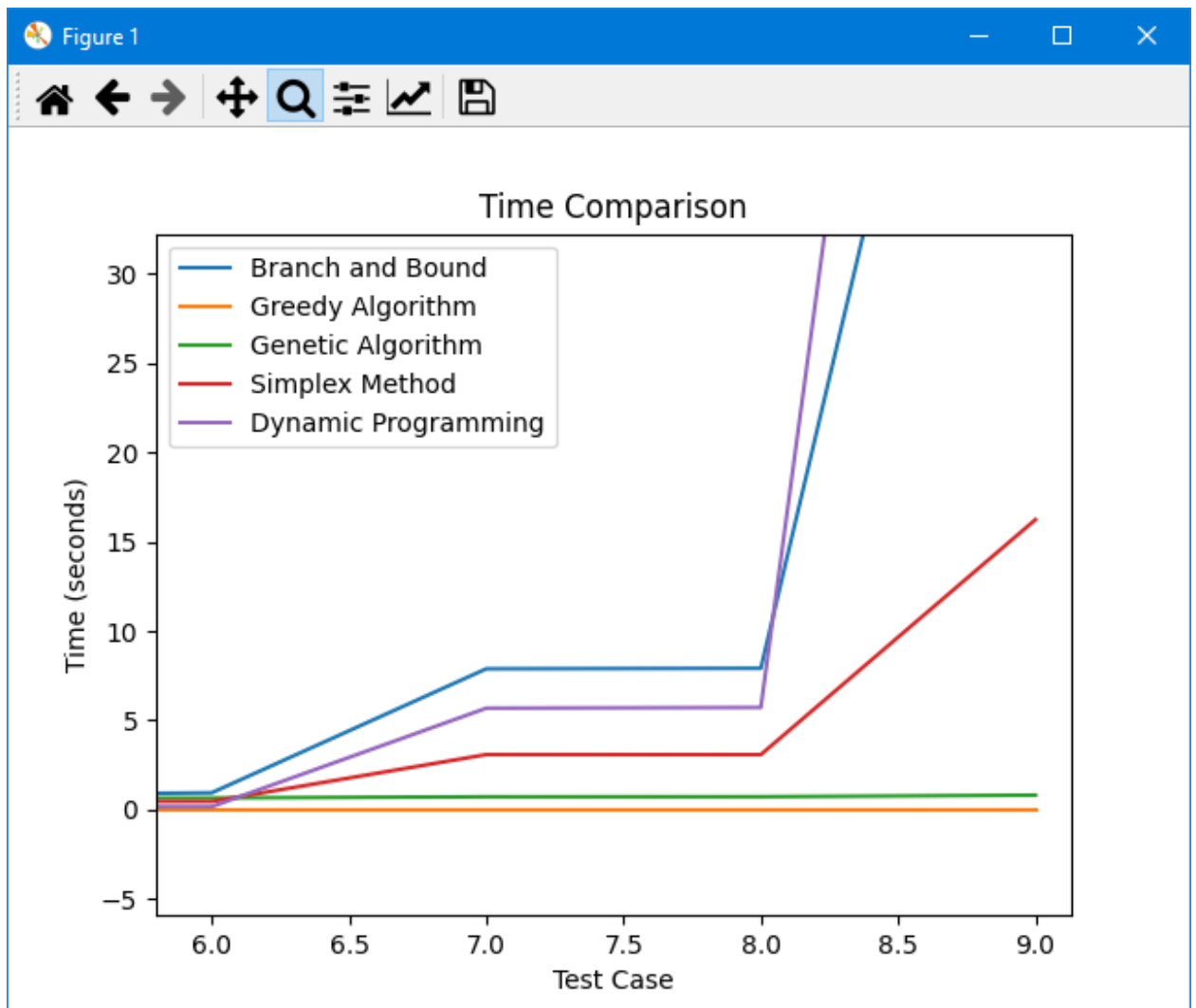


Рисунок 29 – Зависимость времени от количества переменных в приближении

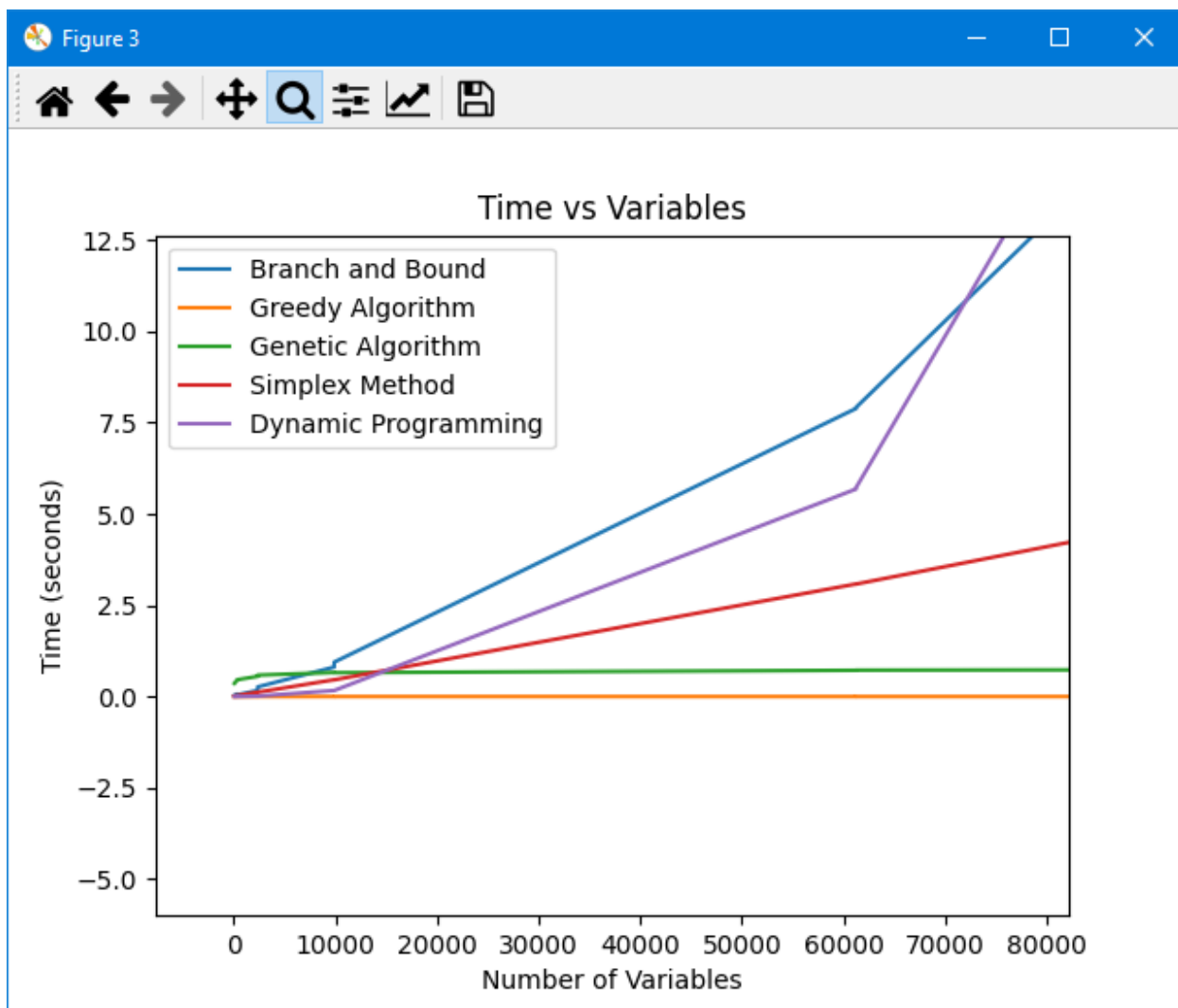


Рисунок 30 – Зависимость времени от количества тестов в приближении

Жадный алгоритм, не смотря на сохранение быстроты выполнения (8 микросекунд), на данном этапе дает слишком далекие от оптимальных значений решения.

4.4 Вывод по проведенным исследованиям

В конечном итоге эмпирические исследования лишь подтвердили теоретические доводы, и полиномиальная сложность алгоритма динамического программирования делает его совсем неэффективным при работе с большими данными. Аналоги в виде симплекс-метода и генетического алгоритма показали себя намного лучше в задачах подобной

размерности, что видно из проведенных тестов. Однако не все оптимизационные задачи можно свести к задачам линейного программирования. Даже усложнение текущей задачи в виде добавления нелинейных функций для расчета прибыли предприятий сделает применение симплекс-метода невозможным. Для генетических алгоритмов тоже могут возникнуть некоторые трудности, ведь он очень чувствителен к правильному выбору фитнес функции и формированию стартовой популяции. В свою очередь, если решаемая задача удовлетворяет принципу оптимальности Беллмана и обладает нужными характеристиками, то метод динамического программирования становится наиболее предпочтительным. Ведь паттерн гарантирует оптимальное решение за разумное время при «адекватной» размерности задачи, что также было продемонстрировано в тестах. При количестве переменных менее 10000, простейшая реализация метода динамического программирования (без использования специальных структур и особенностей языка для сокращения времени выполнения), смогла превзойти готовые рыночные решения, над разработкой которых трудились не один год, обгоняя по эффективности времени выполнения представленные алгоритмы в 2-3 раза со средним временем выполнения менее секунды. Поэтому проведенное исследование лишь указало на актуальность и валидность использования метода динамического программирования для решения задач перспективного планирования.

ЗАКЛЮЧЕНИЕ

Цель выпускной квалификационной работы – автоматизация решения задачи оптимального инвестирования на основе динамического программирования – достигнута.

Решены следующие задачи:

- осуществлена формализация задачи оптимального распределения ресурсов и алгоритмизация ее решения;
- разработан проект и соответствующая программа, реализующая метод динамического программирования;
- проведено численное моделирование полученного ПО на тестовых примерах;
- исследованы результаты решения в зависимости от начальных данных;
- проведены сравнение эффективности работы алгоритма с существующими аналогами и анализ полученных результатов.

Разработанная программа может быть использована для решения задачи распределения ресурсов конечным пользователем и проведения исследования различных сценариев инвестирования.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Козинченко Е., Мордовенко Д., Шехад Ж., Тидеман Д. Инвестиционные проекты в российской нефтегазовой отрасли: четыре шага к повышению эффективности [Электронный ресурс] // Экономика и управление: актуальные проблемы и поиск путей решения. – URL: <http://econom.psu.ru/upload/iblock/501/ekonomika-i-upravlenie-2017.pdf> (дата обращения 28.04.2024).
2. Кархов В.А. Инвестиционная деятельность нефтегазовых компаний [Электронный ресурс]. – URL: izdatelstvo.isea.ru>epm/dl.ashx?id=2505 (дата обращения 28.04.2024)
3. Набатова, Д. С. Математические и инструментальные методы поддержки принятия решений [Текст] / Д. С. Набатова — Москва: Юрайт, 2023 — 293 с.
4. АБСАЛЯМОВ Д. Р., ХАЛЬМЕТОВ Р. Р., ПОЛЯКОВ С. А. ПОДХОДЫ К ФОРМИРОВАНИЮ ЗИП ПРИ ЭКСПЛУАТАЦИИ СИСТЕМ ЖИЗНЕОБЕСПЕЧЕНИЯ ОБЪЕКТОВ НАЗЕМНОЙ КОСМИЧЕСКОЙ ИНФРАСТРУКТУРЫ //Актуальные проблемы военно-научных исследований. – 2020. – №. S10. – С. 44-53.
5. Динамическое программирование / [Электронный ресурс] // Википедия: [сайт]. — URL: https://ru.wikipedia.org/wiki/Динамическое_программирование (дата обращения: 01.05.2024).
6. Сутягина Н. И. Метод динамического программирования при принятии микроэкономического решения [Текст] / Сутягина Н. И. // Вестник НГИЭИ. — 2014. — № 11. — С. 72-77.
7. Ayat Rakhishev Dynamic Programming, или как использовать предыдущий computation-опыт / Ayat Rakhishev [Электронный ресурс] // Хабр : [сайт]. — URL: <https://habr.com/ru/companies/inDrive/articles/711352/> (дата обращения: 03.05.2024).

8. Корнеев А. М., Лаврухина Т. В., Сметанникова Т. А. Решение задачи оптимального распределения ресурсов МЧС методом динамического программирования //Тенденции развития науки и образования. – 2021. – №. 71-1. – С. 20-24.
9. Ломкова Е. Н., Эпов А. А.. Экономико-математические модели управления производством (теоретические аспекты): Учеб. пособие / ВолгГТУ, Волгоград, 2005. – 67 с
10. Акулич И. Л. Математическое программирование в примерах и задачах. – Высшая школа, 1993. – С. 336-336.
11. Вентцель Е. С. Элементы динамического программирования. – 1964.
12. Богомолова И.В. Математические методы в экономике: учеб.-метод. пособие / И.В. Богомолова, В.П. Глухов, А.М. Лебедев. - Ульяновск: УВАУ ГА, 2005. - 81 с.
13. Глухова Н.В. НОВАЯ МЕТОДИКА ИЗУЧЕНИЯ ТЕМЫ «ДИНАМИЧЕСКОЕ ПРОГРАММИРОВАНИЕ» НА ПРИМЕРЕ ЗАДАЧИ ОБ ИНВЕСТИРОВАНИИ ДЛЯ СТУДЕНТОВ, ОБУЧАЮЩИХСЯ ЭКОНОМИКЕ И УПРАВЛЕНИЮ // Фундаментальные исследования. – 2014. – № 8-4. – С. 950-954;
14. Калязина С. Е., Дубгорн А. С., Лепехин А. А. ПРИМЕНЕНИЕ ПРОТОТИПИРОВАНИЯ ПРИ РАЗРАБОТКЕ РЕКОМЕНДАТЕЛЬНОЙ СИСТЕМЫ ДЛЯ АНАЛИЗА И ПОПОЛНЕНИЯ СКЛАДСКИХ ЗАПАСОВ //ФУНДАМЕНТАЛЬНЫЕ И ПРИКЛАДНЫЕ ИССЛЕДОВАНИЯ В ОБЛАСТИ УПРАВЛЕНИЯ, ЭКОНОМИКИ И ТОРГОВЛИ. – 2018. – С. 31-36.
15. Richard L. Halterman, Learning to program with Python / Richard L. Halterman, 2011. – 283 p.
16. Python 3.12.3 documentation tkinter — Python interface to Tcl/Tk / Python 3.12.3 documentation [Электронный ресурс] // Python : [сайт]. — URL:

<https://docs.python.org/3/library/tkinter.html#tkinter-modules> (дата обращения: 10.05.2024).

17. Python 3.12.3 documentation json — json — JSON encoder and decoder [Электронный ресурс] // Python : [сайт]. — URL: <https://docs.python.org/3/library/json.html> (дата обращения: 12.05.2024).

18. Coin OR How to configure a solver in PuLP / Coin OR [Электронный ресурс] // PuLP documentation 2.8.0 : [сайт]. — URL: https://coin-or.github.io/pulp/guides/how_to_configure_solvers.html (дата обращения: 15.05.2024).

19. GLPK - GNU Project - Free Software Foundation (FSF) GLPK (GNU Linear Programming Kit) / GLPK - GNU Project - Free Software Foundation (FSF) [Электронный ресурс] // : [сайт]. — URL: <https://www.gnu.org/software/glpk/> (дата обращения: 15.05.2024).

20. DEAP Project Deap documentation / DEAP Project [Электронный ресурс] // DEAP 1.4.1 documentation : [сайт]. — URL: <https://deap.readthedocs.io/en/master/index.html> (дата обращения: 19.05.2024).

ПРИЛОЖЕНИЕ. Текст программы

```
import tkinter as tk
from tkinter import ttk, messagebox, filedialog
import json
import dp_test4

class ResourceAllocationApp:
    def __init__(self, master):
        self.master = master
        self.master.title("Resource Allocation")

        self.resources = []
        self.entries = []

        # Parameters Frame
        params_frame = tk.Frame(master)
        params_frame.grid(row=0, column=0, padx=10, pady=10,
sticky="nw")

        tk.Label(params_frame, text="Ресурс:").grid(row=0, column=0,
sticky="w")

        self.resource_var = tk.DoubleVar(value=100.0)
        tk.Entry(params_frame, textvariable=self.resource_var).grid(row=0,
column=1)

        tk.Label(params_frame, text="Количество вариантов эффективного
вложения:").grid(row=1, column=0, sticky="w")

        self.var_count = tk.IntVar(value=6)
        self.var_count_menu = tk.OptionMenu(params_frame, self.var_count,
*[i for i in range(1, 11)], command=self.update_table)
```

```

self.var_count_menu.grid(row=1, column=1)

tk.Label(params_frame, text="Количество
предприятий:").grid(row=2, column=0, sticky="w")
self.ent_count = tk.IntVar(value=4)
self.ent_count_menu = tk.OptionMenu(params_frame, self.ent_count,
*[i for i in range(1, 11)], command=self.update_table)
self.ent_count_menu.grid(row=2, column=1)

tk.Button(params_frame, text="Распределить",
command=self.allocate).grid(row=3, column=0, columnspan=2)
tk.Button(params_frame, text="Очистить",
command=self.clear).grid(row=4, column=0, columnspan=2)
tk.Button(params_frame, text="Обновить ресурс",
command=self.update_table).grid(row=5, column=0, columnspan=2)
tk.Button(params_frame, text="Сохранить",
command=self.save_to_file).grid(row=6, column=0, columnspan=2)
tk.Button(params_frame, text="Загрузить",
command=self.load_from_file).grid(row=7, column=0, columnspan=2)

# Table Frame
self.table_frame = tk.Frame(master)
self.table_frame.grid(row=0, column=1, padx=10, pady=10)

# Result Frame
self.result_frame = tk.Frame(master)
self.result_frame.grid(row=1, column=0, columnspan=2, padx=10,
pady=10)

```

```
tk.Label(self.result_frame, text="Оптимальное распределение  
ресурсов по предприятиям").grid(row=0, column=0, columnspan=10)
```

```
self.res_labels = []  
self.res_vars = []  
self.total_income = tk.StringVar(value="0.0")  
self.update_table()
```

```
def update_table(self, *args):  
    # Сохранение текущих значений таблицы  
    current_values = []  
    for row_entries in self.entries:  
        row_values = []  
        for entry in row_entries:  
            try:  
                row_values.append(float(entry.get()))  
            except ValueError:  
                row_values.append(0.0)  
        current_values.append(row_values)  
  
    # Удаление всех виджетов из table_frame  
    for widget in self.table_frame.winfo_children():  
        widget.destroy()  
  
    # Создание новой таблицы  
    col_count = self.ent_count.get() + 1  
    row_count = self.var_count.get() + 1  
  
    self.entries = []
```

```

for i in range(row_count):
    row_entries = []
    for j in range(col_count):
        if i == 0:
            if j == 0:
                label = tk.Label(self.table_frame, text="Ресурсы")
            else:
                label = tk.Label(self.table_frame, text=f"# {j}")
            label.grid(row=i, column=j)
        else:
            if j == 0:
                label = tk.Label(self.table_frame, text=f"{{(i-1) *
self.resource_var.get() / (row_count - 2):.1f}}")
            label.grid(row=i, column=j)
        else:
            entry = tk.Entry(self.table_frame)
            # Восстановление значений из текущих значений
таблицы, если они существуют
            if i - 1 < len(current_values) and j - 1 < len(current_values[i -
1]):
                entry.insert(0, f"{{current_values[i - 1][j - 1]:.1f}}")
            else:
                entry.insert(0, "0.0")
            entry.grid(row=i, column=j)
            row_entries.append(entry)
    if row_entries:
        self.entries.append(row_entries)
# Обновление таблицы с результатами

```

```
self.update_result_table()
```

```
def update_result_table(self):  
    for widget in self.result_frame.winfo_children():  
        widget.destroy()
```

```
tk.Label(self.result_frame, text="Оптимальное распределение  
ресурсов по предприятиям").grid(row=0, column=0, columnspan=10)
```

```
self.res_labels = []  
self.res_vars = []  
col_count = self.ent_count.get()  
for i in range(col_count):  
    label = tk.Label(self.result_frame, text=f"# {i+1}")  
    label.grid(row=1, column=i)  
    self.res_labels.append(label)
```

```
self.res_vars = [tk.StringVar(value="0.0") for _ in range(col_count)]  
for i, var in enumerate(self.res_vars):  
    label = tk.Label(self.result_frame, textvariable=var)  
    label.grid(row=2, column=i)
```

```
tk.Label(self.result_frame, text="Суммарный доход:").grid(row=3,  
column=0, columnspan=3, sticky="e")  
tk.Label(self.result_frame, textvariable=self.total_income).grid(row=3,  
column=3, sticky="w")
```

```
def allocate(self):
```

```

# Check if all entries are filled
try:
    self.resource_var.get()
except tk.TclError:
    messagebox.showerror("Ошибка", "Укажите корректное значение
ресурса")
    return

values = []
try:
    for row in self.entries:
        row_values = [float(entry.get()) for entry in row]
        values.append(row_values)
except ValueError:
    messagebox.showerror("Ошибка", "Заполните все ячейки
корректными числами")
    return

# Logic to allocate resources
# For simplicity, just sum up the first column
# Replace this with the actual optimization logic
# Считывание значения ресурса
S = self.resource_var.get()

# Считывание значений из основной таблицы в матрицу profit[][]
profit = []
for row_entries in self.entries:
    row_values = []
    for entry in row_entries:

```

```

    try:
        row_values.append(float(entry.get()))
    except ValueError:
        row_values.append(0.0)
    profit.append(row_values)

# Вызов функции динамического программирования
max_profit, result = dp_test4.dynamic_programming_2(S, profit)

# Обновление результирующей таблицы
for j in range(self.ent_count.get()):
    self.res_vars[j].set(f'{result[j]:.1f}')

# Обновление суммарного дохода
self.total_income.set(f'{max_profit:.1f}')

def clear(self):
    self.resource_var.set(100.0)
    for row in self.entries:
        for entry in row:
            entry.delete(0, tk.END)
            entry.insert(0, "0.0")
    for var in self.res_vars:
        var.set("0.0")
    self.total_income.set("0.0")

def save_to_file(self):
    save_data = {
        "matrix": [[entry.get() for entry in row] for row in self.entries],

```

```

        "resource": self.resource_var.get(),
        "var_count": self.var_count.get(),
        "ent_count": self.ent_count.get()
    }

    file_path = filedialog.asksaveasfilename(defaultextension=".json",
filetypes=[("JSON files", "*.json")])

    if file_path:
        with open(file_path, 'w') as file:
            json.dump(save_data, file)

        messagebox.showinfo("Уведомление", "Ваши данные были
успешно сохранены!")

```

```

def load_from_file(self):
    file_path = filedialog.askopenfilename(filetypes=[("JSON files",
"*.json")])

    if file_path:
        with open(file_path, 'r') as file:
            load_data = json.load(file)

            self.var_count.set(load_data["var_count"])
            self.ent_count.set(load_data["ent_count"])
            self.resource_var.set(load_data["resource"])
            self.update_table()

            for i, row in enumerate(load_data["matrix"]):
                for j, value in enumerate(row):
                    self.entries[i][j].delete(0, tk.END)
                    self.entries[i][j].insert(0, value)

```

```
        messagebox.showinfo("Уведомление", "Ваши данные были  
успешно загружены!")
```

```
class DriveTestApp:  
    def __init__(self) -> None:  
        pass
```

```
class MainApp:  
    def __init__(self, root):  
        self.root = root  
        self.root.title("Main Menu")
```

```
        tk.Button(root, text="Оптимальное распределение ресурсов",  
command=self.open_resource_allocation).pack(padx=10, pady=10)  
        # Add other buttons for different programs here  
        tk.Button(root, text="Сравнение алгоритмов",  
command=self.open_test_algorithms).pack(padx=10, pady=10)  
    def open_resource_allocation(self):  
        new_window = tk.Toplevel(self.root)  
        ResourceAllocationApp(new_window)  
    def open_test_algorithms(self):  
        new_window = tk.Toplevel(self.root)  
        DriveTestApp(new_window)
```

```
if __name__ == "__main__":  
    root = tk.Tk()  
    app = MainApp(root)  
    root.mainloop()
```

```

#dp_test4 file

import random
import math
import time
import psutil
import pulp
from deap import base, creator, tools, algorithms
import matplotlib.pyplot as plt
from tabulate import tabulate
import numpy as np

def generate_random_data(S, N, delta):
    profit = [[random.randint(0, 10) + (j + 1) for j in range(S // delta + 1)] for i
in range(N)]
    return profit

def branch_and_bound(S, N, delta, profit):
    print("Testing branch_and_bound...")
    # Создание проблемы
    prob = pulp.LpProblem("Maximize_Profit", pulp.LpMaximize)

    # Переменные
    x = pulp.LpVariable.dicts("x", (range(N), range(S // delta + 1)),
cat='Binary')

    # Целевая функция
    prob += pulp.lpSum(profit[i][j] * x[i][j] for i in range(N) for j in range(S
// delta + 1)), "Total_Profit"

```

```

# Ограничение на общие средства
prob += pulp.lpSum(j * delta * x[i][j] for i in range(N) for j in range(S //
delta + 1)) == S, "Total_Investment"

# Ограничение на выбор одного уровня инвестиций для каждого
предприятия
for i in range(N):
    prob += pulp.lpSum(x[i][j] for j in range(S // delta + 1)) == 1,
f"One_Level_Investment_{i}"

# Решение проблемы
prob.solve()
print("Status:", pulp.LpStatus[prob.status])
for i in range(N):
    for j in range(S // delta + 1):
        if pulp.value(x[i][j]) > 0:
            print(f"Средства для предприятия {i+1}: {j * delta}, значение
переменной: {pulp.value(x[i][j])}")
    print("Общая прибыль:", pulp.value(prob.objective))

return math.floor(pulp.value(prob.objective))

def greedy_algorithm(S, N, delta, profit):
    # Количество шагов финансирования для каждого предприятия
    max_levels = S // delta + 1

    # Текущие вложения для каждого предприятия
    investments = [0] * N

```

```

# Оставшиеся средства
remaining_budget = S

# Жадный алгоритм распределения средств
while remaining_budget > 0:
    best_increase = 0
    best_choice = None

    for i in range(N):
        if investments[i] < max_levels - 1:
            current_level = investments[i]
            next_level = current_level + 1
            increase = profit[i][next_level] - profit[i][current_level]

            if increase > best_increase:
                best_increase = increase
                best_choice = (i, next_level)

    if best_choice is None:
        break

    i, next_level = best_choice
    investments[i] = next_level
    remaining_budget -= delta

max_profit = sum(profit[i][investments[i]] for i in range(N))
print("Максимальная прибыль равна: ", max_profit)
print("Распределение вложений по предприятиям:", investments)
return max_profit

```

```

def genetic_algorithm(S, N, delta, profit):
    # Количество шагов финансирования для каждого предприятия
    max_levels = S // delta + 1

    # Создание типов данных для задачи
    creator.create("FitnessMax", base.Fitness, weights=(1.0,))
    creator.create("Individual", list, fitness=creator.FitnessMax)

    toolbox = base.Toolbox()
    toolbox.register("attr_int", random.randint, 0, max_levels - 1)
    # Инициализация индивида с ограничением суммы вложений
    def init_individual():
        individual = []
        path = S // delta
        step = random.randint(1, max_levels - 1)
        path = path - step
        individual.append(path)
        while path >= 0:
            step = random.randint(1, max_levels - 1)
            path = path - step
            if path >= 0:
                individual.append(path)
        length = len(individual)
        while length < N:
            individual.append(0)
            length+=1
        return individual

```

```

        toolbox.register("individual", tools.initIterate, creator.Individual,
init_individual)

        toolbox.register("population", tools.initRepeat, list, toolbox.individual)

# Оценочная функция
def evalProfit(individual):
    total_investment = sum(individual) * delta
    if total_investment > S:
        return -float('inf'), # Штраф за превышение бюджета
    total_profit = sum(profit[i][individual[i]] for i in range(N))
    return total_profit,

toolbox.register("evaluate", evalProfit)
toolbox.register("mate", tools.cxTwoPoint)
toolbox.register("mutate", tools.mutUniformInt, low=0, up=max_levels -
1, indpb=0.1)
toolbox.register("select", tools.selTournament, tournsize=5)

# Параметры генетического алгоритма
population_size = 300
prob_crossover = 0.65
prob_mutation = 0.35
num_generations = 50

# Инициализация популяции
population = toolbox.population(n=population_size)

# Статистика
stats = tools.Statistics(lambda ind: ind.fitness.values)

```

```

stats.register("avg", lambda x: sum(val[0] for val in x) / len(x))
stats.register("min", lambda x: min(val[0] for val in x))
stats.register("max", lambda x: max(val[0] for val in x))

# Генетический алгоритм
algorithms.eaSimple(population, toolbox, prob_crossover, prob_mutation,
num_generations, stats=stats)

# Вывод результатов
best_individual = tools.selBest(population, 1)[0]
return evalProfit(best_individual)[0]

def simplex_method(S, N, delta, profit):
    print("Testing simplex_method...")
    # Создание проблемы
    prob = pulp.LpProblem("Maximize_Profit", pulp.LpMaximize)

    # Переменные
    x = pulp.LpVariable.dicts("x", (range(N), range(S // delta + 1)),
lowBound=0, upBound=1, cat='Continuous')

    # Целевая функция
    prob += pulp.lpSum(profit[i][j] * x[i][j] for i in range(N) for j in range(S
// delta + 1)), "Total_Profit"

    # Ограничение на общие средства
    prob += pulp.lpSum(j * delta * x[i][j] for i in range(N) for j in range(S //
delta + 1)) <= S, "Total_Investment"

```

```

# Ограничение на выбор одного уровня инвестиций для каждого
предприятия
for i in range(N):
    prob += pulp.lpSum(x[i][j] for j in range(S // delta + 1)) == 1,
    f"One_Level_Investment_{i}"

# Решение проблемы с использованием GLPK
prob.solve(pulp.GLPK_CMD(path = 'winglpk-4.65/glpk-
4.65/w64/glpso.exe', options=['--simplex']))
for i in range(N):
    for j in range(S // delta + 1):
        if pulp.value(x[i][j]) > 0:
            print(f"Средства для предприятия {i+1}: {j * delta}, значение
переменной: {pulp.value(x[i][j])}")
    print("Общая прибыль:", pulp.value(prob.objective))
    return math.floor(pulp.value(prob.objective))

def dynamic_programming(S, profit_transparent):
    print("Testting DP...")
    # changable
    factories_num = len(profit_transparent)
    steps_num = len(profit_transparent[0])
    delta = S/(steps_num-1)
    calculated_profit = [[0 for _ in range(steps_num)] for _ in
range(factories_num)]
    transitions_matrix = [[0 for _ in range(steps_num)] for _ in
range(factories_num)]

    for cur_step in range(steps_num):

```

```

calculated_profit[0][cur_step] = profit_transparent[0][cur_step]
transitions_matrix[0][cur_step] = cur_step

for cur_fact in range(1,factories_num):
    for cur_step in range(steps_num):
        step_profit_matrix = [0 for _ in range(cur_step+1)]
        for x in range(cur_step+1):
            step_profit_matrix[x] = profit_transparent[cur_fact][x] +
calculated_profit[cur_fact-1][cur_step-x]
            cur_max_profit = max(step_profit_matrix)
                                transitions_matrix[cur_fact][cur_step] =
step_profit_matrix.index(cur_max_profit)
            calculated_profit[cur_fact][cur_step] = cur_max_profit

max_profit = max(calculated_profit[factories_num-1])
print("Максимальная прибыль равна: ",max_profit)

result = [0 for _ in range(factories_num)]
        optimal_transition = calculated_profit[factories_num-
1].index(max_profit)
    for step in range(factories_num-1, -1, -1):
        optimal_step = transitions_matrix[step][optimal_transition]
        result[step] = optimal_step * delta
        optimal_transition = optimal_transition - optimal_step

print("Распределение вложений по предприятиям:", result)
return max_profit

def dynamic_programming_2(S,profit):

```

```

# changable
profit_transparent = list(map(list, zip(*profit)))
factories_num = len(profit_transparent)
steps_num = len(profit_transparent[0])
delta = S/(steps_num-1)

    calculated_profit = [[0 for _ in range(steps_num)] for _ in
range(factories_num)]

    transitions_matrix = [[0 for _ in range(steps_num)] for _ in
range(factories_num)]

for cur_step in range(steps_num):
    calculated_profit[0][cur_step] = profit_transparent[0][cur_step]
    transitions_matrix[0][cur_step] = cur_step

for cur_fact in range(1,factories_num):
    for cur_step in range(steps_num):
        step_profit_matrix = [0 for _ in range(cur_step+1)]
        for x in range(cur_step+1):
            step_profit_matrix[x] = profit_transparent[cur_fact][x] +
calculated_profit[cur_fact-1][cur_step-x]
            cur_max_profit = max(step_profit_matrix)
            transitions_matrix[cur_fact][cur_step] =
step_profit_matrix.index(cur_max_profit)
            calculated_profit[cur_fact][cur_step] = cur_max_profit

max_profit = max(calculated_profit[factories_num-1])
print("Максимальная прибыль равна: ",max_profit)

result = [0 for _ in range(factories_num)]

```

```

all_steps = steps_num-1
    optimal_transition = calculated_profit[factories_num-
1].index(max_profit)
    for step in range(factories_num-1, -1, -1):
        optimal_step = transitions_matrix[step][optimal_transition]
        result[step] = optimal_step * delta
        optimal_transition = all_steps - optimal_step
        all_steps = all_steps - optimal_step

    print("Распределение вложений по предприятиям:", result)
    return max_profit, result

def compare_algorithms(test_cases):
    results = {
        "Branch and Bound": [],
        "Greedy Algorithm": [],
        "Genetic Algorithm": [],
        "Simplex Method": [],
        "Dynamic Programming": []
    }

    for S, N, delta, profit in test_cases:
        print(f"Testing with S={S}, N={N}, delta={delta}")
        variables_num = len(profit)*len(profit[0])
        # Branch and Bound
        start_time = time.time()
        start_mem = psutil.Process().memory_info().rss
        max_profit = branch_and_bound(S, N, delta, profit)
        end_mem = psutil.Process().memory_info().rss

```

```

end_time = time.time()
        results["Branch and Bound"].append((end_time - start_time,
max_profit, end_mem - start_mem, variables_num))

# Greedy Algorithm
start_time = time.time()
start_mem = psutil.Process().memory_info().rss
max_profit = greedy_algorithm(S, N, delta, profit)
end_mem = psutil.Process().memory_info().rss
end_time = time.time()
        results["Greedy Algorithm"].append((end_time - start_time,
max_profit, end_mem - start_mem, variables_num))

# Genetic Algorithm
start_time = time.time()
start_mem = psutil.Process().memory_info().rss
max_profit = genetic_algorithm(S, N, delta, profit)
end_mem = psutil.Process().memory_info().rss
end_time = time.time()
        results["Genetic Algorithm"].append((end_time - start_time,
max_profit, end_mem - start_mem, variables_num))

# Simplex Method
start_time = time.time()
start_mem = psutil.Process().memory_info().rss
max_profit = simplex_method(S, N, delta, profit)
end_mem = psutil.Process().memory_info().rss
end_time = time.time()

```

```

        results["Simplex Method"].append((end_time - start_time, max_profit,
end_mem - start_mem, variables_num))

```

```

    # Dynamic Programming

```

```

    start_time = time.time()

```

```

    start_mem = psutil.Process().memory_info().rss

```

```

    max_profit = dynamic_programming(S, profit)

```

```

    end_mem = psutil.Process().memory_info().rss

```

```

    end_time = time.time()

```

```

        results["Dynamic Programming"].append((end_time - start_time,
max_profit, end_mem - start_mem, variables_num))

```

```

    return results

```

```

def plot_multiple_results(results):

```

```

    labels = list(results.keys())

```

```

    times = [[result[0] for result in results[label]] for label in labels]

```

```

    profits = [[result[1] for result in results[label]] for label in labels]

```

```

    memories = [[result[2] for result in results[label]] for label in labels]

```

```

    fig, axs = plt.subplots(3, 1, figsize=(10, 12))

```

```

    # Time comparison

```

```

    for label, time_data in zip(labels, times):

```

```

        axs[0].plot(time_data, label=label)

```

```

    axs[0].set_title('Time Comparison')

```

```

    axs[0].set_xlabel('Test Case')

```

```

    axs[0].set_ylabel('Time (seconds)')

```

```

    axs[0].legend()

```

```

# Profit comparison
for label, profit_data in zip(labels, profits):
    axs[1].plot(profit_data, label=label)
axs[1].set_title('Profit Comparison')
axs[1].set_xlabel('Test Case')
axs[1].set_ylabel('Profit')
axs[1].legend()

# Memory comparison
for label, memory_data in zip(labels, memories):
    axs[2].plot(memory_data, label=label)
axs[2].set_title('Memory Usage Comparison')
axs[2].set_xlabel('Test Case')
axs[2].set_ylabel('Memory (bytes)')
axs[2].legend()

plt.tight_layout()
plt.show()

def plot_time_results(results):
    plt.figure()
    labels = list(results.keys())
    times = [[result[0] for result in results[label]] for label in labels]

    for label, time_data in zip(labels, times):
        plt.plot(time_data, label=label)

plt.title('Time Comparison')

```

```
plt.xlabel('Test Case')
plt.ylabel('Time (seconds)')
plt.legend()
```

```
def plot_profit_results(results):
    plt.figure()
    labels = list(results.keys())
    profits = [[result[1] for result in results[label]] for label in labels]

    for label, profit_data in zip(labels, profits):
        plt.plot(profit_data, label=label)

    plt.title('Profit Comparison')
    plt.xlabel('Test Case')
    plt.ylabel('Profit')
    plt.legend()
```

```
def plot_memory_results(results):
    plt.figure()
    labels = list(results.keys())
    memories = [[result[2] for result in results[label]] for label in labels]

    for label, memory_data in zip(labels, memories):
        plt.scatter(range(len(memory_data)), memory_data, label=label)

    plt.title('Memory Usage Comparison')
    plt.xlabel('Test Case')
    plt.ylabel('Memory (bytes)')
```

```
plt.legend()
```

```
def plot_time_vs_variables(results):  
    plt.figure()  
    labels = list(results.keys())  
    variables = [[result[3] for result in results[label]] for label in labels]  
    times = [[result[0] for result in results[label]] for label in labels]  
  
    for label, var_data, time_data in zip(labels, variables, times):  
        plt.plot(var_data, time_data, label=label)  
  
    plt.title('Time vs Variables')  
    plt.xlabel('Number of Variables')  
    plt.ylabel('Time (seconds)')  
    plt.legend()
```

```
def plot_variables(results):  
    plt.figure()  
    variables = [result[3] for result in next(iter(results.values()))]  
  
    plt.scatter(range(len(variables)), variables, label='Variables')  
  
    plt.title('Number of Variables Increase')  
    plt.xlabel('Test Case')  
    plt.ylabel('Number of Variables')  
    plt.legend()  
    plt.show()
```

```

from tabulate import tabulate

def print_results(results):
    for algorithm, data in results.items():
        print(f'Results for {algorithm}:')
        headers = ["Test Case", "Execution Time (s)", "Max Profit", "Number
of Variables"]
        table = [[i+1, result[0], result[1], result[3]] for i, result in
enumerate(data)]
        print(tabulate(table, headers=headers, tablefmt="grid"))
        print("\n")
        print("Average time (s):", sum(result[0] for result in data) / len(data))
        print("\n")
        # Extract number of variables and execution times
        num_vars = [result[3] for result in data]
        execution_times = [result[0] for result in data]
        # Calculate linear regression coefficient
        regression_coefficient =
calculate_linear_regression_coefficient(num_vars, execution_times)
        print("Regression Coefficient:", regression_coefficient)
        print("\n")

def calculate_linear_regression_coefficient(x, y):
    A = np.vstack([x, np.ones(len(x))]).T
    m, c = np.linalg.lstsq(A, y, rcond=None)[0]
    return m

"""

```

```

for i in range(num_tests):
    if (i > 0) and (i % 2 == 0):
        S, N = S * 2, N + 2
        delta = delta - 20
        profit = generate_random_data(S,N,delta)
        test_cases.append([S,N,delta,profit])
"""

def test3():
    # Генерация случайных тестовых случаев
    num_tests = 10
    test_cases = []
    delta = 10
    S = 100
    N = 5
    for i in range(num_tests):
        if (i > 0) and (i % 2 != 0):
            S, N = S * random.randint(1,5) * (i+1), N + (i+1) +
random.randint(2,10)
            delta = delta * (i+1)
            profit = generate_random_data(S,N,delta)
            test_cases.append([S,N,delta,profit])
    # Сравнение алгоритмов
    results = compare_algorithms(test_cases)

    #Вывод результатов в консоль в виде таблицы
    print_results(results)

    # Построение графиков

```

```
plot_time_results(results)
plot_profit_results(results)
plot_time_vs_variables(results)
plot_variables(results)
plt.show()
```

```
def test2():
```

```
    # Генерация случайных тестовых случаев
```

```
    num_tests = 8
```

```
    test_cases = []
```

```
    delta = 120
```

```
    S = 240
```

```
    N = 10
```

```
    for i in range(num_tests):
```

```
        if (i > 0) and (i % 2 == 0):
```

```
            S, N = S * 4, N + 4
```

```
            delta = delta - 20
```

```
            profit = generate_random_data(S,N,delta)
```

```
            test_cases.append([S,N,delta,profit])
```

```
    # Сравнение алгоритмов
```

```
    results = compare_algorithms(test_cases)
```

```
    #Вывод результатов в консоль в виде таблицы
```

```
    print_results(results)
```

```
    # Построение графиков
```

```
    plot_time_results(results)
```

```
    plot_profit_results(results)
```

```
    plot_time_vs_variables(results)
```

```

plot_variables(results)

plt.show()

def test1():
    # Генерация случайных тестовых случаев
    num_tests = 6
    test_cases = []
    delta = 100
    S = 200
    N = 10

    #test_cases = [(random.randint(5, 15), random.randint(2, 5), 1,
generate_random_data(random.randint(5, 15), random.randint(2, 5), 1)) for _ in
range(num_tests)]

    #  $S, N = S * \text{random.randint}(1,9) * (i+1), N + (i+1) +$ 
    random.randint(2,10)
    delta = delta * (i+1)

    for i in range(num_tests):
        if (i > 0) and (i % 2 != 0):
            S, N = S+175, N + 2
            delta = delta - 25

            profit = generate_random_data(S,N,delta)
            test_cases.append([S,N,delta,profit])

    # Сравнение алгоритмов
    results = compare_algorithms(test_cases)

    #Вывод результатов в консоль в виде таблицы
    print_results(results)

    # Построение графиков

```

```
plot_time_results(results)
plot_profit_results(results)
plot_time_vs_variables(results)
plot_variables(results)
plt.show()
```

```
#test1()
```

```
#test2()
```

```
#test3()
```